

Python (programlama dili)

Python, önemli girinti kullanımıyla kod okunabilirliğine , sadeliğe ve yazma kolaylığına önem veren , yüksek seviyeli , genel amaçlı bir programlama dilidir .^[38] Kapsamlı ("piller dahil") bir standart kütüphane ve çöp toplama özelliği de mevcuttur. Python, nesne yönelimli programlama ve dinamik tiplere vurgu yaparak birden fazla programlama paradigmasını destekler .

Guido van Rossum, 1980'lerin sonlarında ABC programlama dilinin halefi olarak Python üzerinde çalışmaya başladı. 2008'de piyasaya sürülen Python 3.0, büyük bir revizyondur ve önceki sürümlerle tamamen geriye dönük uyumlu değildir . Python 3.5'ten itibaren^[39] , isteğe bağlı statik tiplere izin veren, tiplere için yetenekler ve anahtar kelimeler dille eklendi .^[40] 2026 itibarıyla Python Yazılım Vakfı , projenin yıllık sürüm döngüsünü ve beş yıllık destek politikasını takip ederek Python 3.10, 3.11, 3.12, 3.13 ve 3.14'ü desteklemektedir. Python 3.15 şu anda beta geliştirme aşamasındadır ve kararlı sürümün Ekim 2026'da piyasaya sürülmesi beklenmektedir.^[41] 3.x serisindeki önceki sürümler kullanım ömrünün sonuna ulaşmıştır ve artık güvenlik güncellemeleri almamaktadır.

Python, yaygın olarak giriş seviyesi bir programlama dili olarak öğretilmektedir.^[42]

Tarih

Python, 1980'lerin sonlarında^[11] Hollanda'daki Centrum Wiskunde & Informatica'da (CWI) Guido van Rossum tarafından tasarlandı .^[43] SETL'den^[44] esinlenen , istisna işleme yeteneğine sahip ve Amoeba işletim sistemiyle arayüz oluşturabilen ABC programlama dilinin halefi olarak tasarlandı .^[18] Python uygulamasına Aralık 1989'da başlandı.^[43] Van Rossum, ilk olarak 1991'de Python 0.9.0 olarak yayınladı.^[43] Van Rossum, 12 Temmuz 2018'e kadar projenin baş geliştiricisi olarak tek başına

Python



Paradigma	Çoklu paradigma : nesne yönelimli , ^[1] prosedürel (zorunlu) , işlevsel , yapılandırılmış , yansıtıcı
Tasarımcı :	Guido van Rossum
Geliştirici	Python Yazılım Vakfı
İlk olarak yayınlandı	20 Şubat 1991 ^[2]
Kararlı sürüm	3.14.6 ^[3] / 10 Haziran 2026
Yazma disiplini	Ördek , dinamik , güçlü ; ^[4] isteğe bağlı tür açıklamaları ^[a]
Bellek yönetimi	Çöp toplama
İşletim sistemi	Çapraz platform ^[b]
Lisans	Python Yazılım Vakfı Lisansı
Dosya adı uzantıları	.py, ^[11] .pyc, ^[12] .pyd, ^[13] .pyi, ^[14] .pyw, ^[15] .pyz ^[16]
Web sitesi	python.org (https://www.python.org/)

Büyük uygulamalar

CPython , PyPy , MicroPython , CircuitPython , IronPython , Jython , Stackless Python

Lehçeler

Cython , RPython , Starlark^[17]

Etkilendiğim

ABC ,^[18] Ada ,^[19] ALGOL 68 ,^[20] APL ,^[21] C ,^[22] C++ ,^[23] CLU ,^[24] Dylan ,^[25] Haskell ,^[26]^[21] Icon ,^[27] Lisp ,

sorumluluğu üstlendi; bu tarihte Python'ın " ömür boyu hayırsever diktatörü " (BDFL) olarak sorumluluklarından "kalıcı olarak tatile" çıktığını duyurdu; bu unvan, projenin baş karar vericisi olarak uzun süreli bağlılığını yansıtmak için Python topluluğu tarafından kendisine verildi. ^[45]^[c] Ocak 2019'da, aktif Python çekirdek geliştiricileri projeye liderlik edecek beş üyeli bir Yönlendirme Konseyi seçti. ^[46]^[47]

^[28] Modula-3 , ^[20]^[23] Perl , ^[29] Standard ML ^[21]

Etkilendi

Apache Groovy , Boo , Cobra , CoffeeScript , ^[30] D , F# , GDScript , Go , JavaScript , ^[31]^[32] Julia , ^[33] Mojo , ^[34] Nim , Ruby , ^[35] Swift , ^[36] V ^[37]



Vikikitap'ta Python Programlama

Python adı, İngiliz komedi dizisi *Monty Python's Flying Circus'tan* türemiştir . ^[48] (Bkz. § Adlandırma .)

Python 2.0, liste anlama , döngü algılayan çöp toplama, referans sayımı ve Unicode desteği gibi birçok yeni özellik ile 16 Ekim 2000'de piyasaya sürüldü . ^[49] Python 2.7'nin kullanım ömrünün sonu başlangıçta 2015 olarak belirlenmişti, ancak mevcut kodun büyük bir bölümünün Python 3'e kolayca aktarılamayacağı endişesiyle 2020'ye ertelendi. ^[50]^[51] Artık güvenlik yamaları veya güncellemeler almıyor. ^[52]^[53] Python 2.7 ve daha eski sürümler resmi olarak desteklenmezken, farklı bir gayri resmi Python uygulaması olan PyPy , Python 2'yi, yani "2.7.18+" (artı 3.11) sürümlerini desteklemeye devam ediyor ; artı işareti (en azından bazı) " geriye dönük güvenlik güncellemelerini" ifade ediyor. ^[54]



Python'ın tasarımcısı Guido van Rossum , PyCon ABD 2024'te.

Python 3.0, 3 Aralık 2008'de yayınlandı ve önceki sürümlerle tamamen geriye dönük uyumlu olmayan, bazı yeni anlambilim ve değiştirilmiş sözdizimine sahip önemli bir revizyondur. 2020'de yayınlanan Python 2.7.18, Python 2'nin son sürümüydü. ^[55] Python 3.x serisindeki birkaç sürüm, dile yeni sözdizimi ekledi ve birkaç (çok küçük kabul edilen) geriye dönük uyumsuz değişiklik yaptı.

Mayıs 2026 itibarıyla Python 3.14.6 en son kararlı sürümdür. Tüm eski 3.x sürümleri, Python 3.9.24'e kadar güvenlik güncellemesi aldı, ardından 3.9 serisinin son sürümü olan 3.9.25 ile tekrar güncellendi. Python 3.10, Kasım 2025'ten beri desteklenen en eski daldır. ^[56] Python 3.15'in bir alfa sürümü yayınlandı ve Android'de Python 3.14 için resmi olarak indirilebilir bir yürütülebilir dosya mevcut. Sürümler iki yıl tam destek ve ardından üç yıl güvenlik desteği alır.

Tasarım felsefesi ve özellikleri

Python, çoklu paradigmatlı bir programlama dilidir . Nesne yönelimli programlama ve yapısal programlama tamamen desteklenir ve özelliklerinin çoğu, fonksiyonel programlama ve yönelimli programlamayı destekler ; bunlar arasında metaprogramlama ^[57] ve meta nesneler ^[58] bulunur . Sözleşmeye dayalı tasarım ^[59]^[60] ve mantık programlama ^[61] dahil olmak üzere birçok başka paradigma uzantılar aracılığıyla desteklenir . Python, diğer dillerde yazılmış bileşenleri entegre edebilecek şekilde tasarlandığı için sıklıkla *'yapıştırıcı dil'* ^[62] olarak adlandırılır .

Python, bellek yönetimi için dinamik tiplendirme ve referans sayımı ile döngü algılayan çöp toplayıcının bir kombinasyonunu kullanır . [63] Program yürütülmesi sırasında yöntem ve değişken adlarını bağlayan dinamik ad çözümlemesi (geç bağlama) kullanır .

Python'ın tasarımı, " Lispfilter geleneğinde" fonksiyonel programlamaya bazı destekler sunmaktadır. , mapve reducefonksiyonlarına; liste anlama ifadelerine , sözlüklere , kümelere ve üreteç ifadelerine sahiptir . [64] Standart kütüphane, Haskell ve Standard ML'den ödünç alınan fonksiyonel araçları uygulayan iki modüle (itertoolsve) sahiptir . [65] functools

Python'ın temel felsefesi, Tim Peters tarafından yazılan ve aşağıdaki gibi özdeyişleri içeren Python'ın Zen'inde (PEP 20) özetlenmiştir : [66]

- Açık olan, örtülü olandan daha iyidir.
- Basit, karmaşıktan daha iyidir.
- Okunabilirlik önemlidir.
- Özel durumlar, kuralları çiğnemeyi gerektirecek kadar özel değildir.
- Pratiklik saflıktan daha önemli olsa da, hatalar açıkça susturulmadıkça asla göz ardı edilmemelidir.
- Bunu yapmanın tek ve tercihen yalnızca bir tane bariz yolu olmalı.

Ancak Python, bu ilkeleri ihlal ettiği ve gereksiz dil şişkinliği eklediği için eleştiriler aldı. [67] Bu eleştirilere verilen yanıtlar, Python'ın Zen'inin bir kuraldan ziyade bir kılavuz olduğunu belirtiyor. [68] Bazı yeni özelliklerin eklenmesi tartışmalıydı: Guido van Rossum, Python 3.8'e atama ifadesi operatörünün eklenmesiyle ilgili anlaşmazlık nedeniyle *Ömür Boyu Hayırsever Diktatörlükten istifa etti* . [69] [70]

Bununla birlikte, Python tüm işlevselliği çekirdeğine yerleştirmek yerine, modüller aracılığıyla oldukça genişletilebilir olacak şekilde tasarlandı . Bu kompakt modülerlik, onu özellikle mevcut uygulamalara programlanabilir arayüzler eklemenin bir yolu olarak popüler hale getirdi. Van Rossum'un büyük bir standart kütüphaneye ve kolayca genişletilebilir bir yorumlayıcıya sahip küçük bir çekirdek dil vizyonu, tam tersi yaklaşımı temsil eden ABC'ye duyduğu hayal kırıklığından kaynaklanıyordu. [11]

Python, geliştiricilere kodlama metodolojilerinde seçim hakkı verirken, daha basit, daha az karmaşık bir sözdizimi ve dilbilgisi için çabaladığını iddia ediyor . Python'da , Rossum'un zararlı olarak değerlendirdiği do .. whiledöngüler bulunmuyor . [71] Perl'in " bir şeyi yapmanın birden fazla yolu vardır " sloganının aksine , Python "bir şeyi yapmanın tek ve tercihen yalnızca bir açık yolu olmalıdır" yaklaşımını savunuyor. [66] Ancak pratikte Python, belirli bir amaca ulaşmak için birçok yol sunuyor. Bir dize sabitini biçimlendirmenin en az üç yolu vardır ve bir programcının hangisini kullanması gerektiği konusunda kesinlik yoktur. [72] Python Yazılım Vakfı'nda Üye ve Python kitabı yazarı olan Alex Martelli , "Bir şeyi 'akıllı' olarak tanımlamak , Python kültüründe bir iltifat olarak kabul *edilmez* " diye yazdı . [73]

Python geliştiricileri genellikle performanstan ziyade okunabilirliğe öncelik verir. Örneğin, netlik ve okunabilirlik maliyetini haklı çıkarmayan hız artışları sunacak CPython referans uygulamasının kritik olmayan bölümlerine yönelik yamaları reddederler. [74] Yürütme hızı, hız açısından kritik fonksiyonları C gibi dillerde yazılmış uzantı modüllerine taşıyarak veya PyPy gibi anlık derleyici kullanarak iyileştirilebilir . Ayrıca, diğer dillere dönüştürmek de mümkündür. Bununla birlikte, bu yaklaşım, Python çok dinamik bir dil olduğu için beklenen hızlanmayı sağlayamaz veya Python'ın yalnızca kısıtlı bir alt kümesi derlenir (potansiyel küçük anlamsal değişikliklerle). [75]

Python, kullanımı eğlenceli bir dil olmayı amaçlıyor. ^[76]: 3 Bu amaç, İngiliz komedi grubu Monty Python'a bir övgü niteliğindeki isme ^[77] ve bazı öğretici ve referans materyallerine yönelik eğlenceli yaklaşımlara yansıyor. Örneğin, bazı kod örneklerinde tipik "foo" ve "bar" terimleri yerine "spam" ve "eggs" (bir Monty Python skeçine atıfta bulunarak) terimleri kullanılıyor. ^[76]^[78]

Python topluluğunda yaygın bir yeni terim olan "*pythonic*", *program stiliyle ilgili geniş bir anlam yelpazesine sahiptir: Pythonik kod, Python deyimlerini* iyi kullanabilir ; doğal olabilir veya dilde akıcılık gösterebilir; veya Python'ın minimalist felsefesine ve okunabilirliğe verdiği öneme uygun olabilir. ^[79]

Geliştirme Önerileri

Python Geliştirme Önerileri ^[not 1], Python topluluğuna bilgi sağlamak veya Python'da yeni bir özellik önerisi sunmak için kullanılan bir tasarım belgesidir. ^[80] PEP'ler, Python'daki yeni süreçleri açıklamak, adlandırma kuralları sağlamak veya dildeki süreçleri belgelemek amacıyla hazırlanmıştır. ^[81] PEP'ler, Python Yönlendirme Konseyi tarafından denetlenir. ^[81]

Üç çeşit PEP vardır: *Standart izleme PEP'leri* ^[not 2], *Bilgilendirici PEP'ler* ^[not 3] ve *Süreç PEP'leri* ^[not 4] ve bunların her birinin kendine özgü anlamları vardır. ^[80]^[82] Bunlar ilk olarak 2000 yılında, diğer RfC'lerden (yorum talepleri) ve Tasarım Geliştirme Önerilerinden esinlenilerek tanıtılmıştır. ^[82] En bilinen PEP'ler PEP – 1, PEP – 8, PEP – 20, PEP – 257 ve diğerleridir. ^[82]

Sözdizimi ve anlambilim

Python, kolay okunabilir bir dil olarak tasarlanmıştır. Biçimlendirmesi görsel olarak karmaşık değildir ve diğer dillerin noktalama işaretleri kullandığı yerlerde genellikle İngilizce anahtar kelimeler kullanır. Birçok diğer dilin aksine, blokları ayırmak için süslü parantezler kullanmaz ve ifadelerden sonra noktalı virgüllere izin verilir ancak nadiren kullanılır. C veya Pascal'dan daha az sözdizimsel istisna ve özel duruma sahiptir. ^[83]

Girinti

Python, blokları ayırmak için süslü parantezler veya anahtar kelimeler yerine boşluk girintisi kullanır. Girintideki artış belirli ifadelerden sonra gelir; girintideki azalma mevcut bloğun sonunu gösterir. ^[84] Böylece programın görsel yapısı, anlamsal yapısını doğru bir şekilde temsil eder. ^[85] Bu özellik bazen "yan taraf kuralı" olarak adlandırılır. Bazı diğer diller de girintiyi bu şekilde kullanır; ancak çoğunda girintinin anlamsal bir anlamı yoktur. Önerilen girinti boyutu dört boşluktur. ^[86]

Bildirimler ve kontrol akışı

Python'ın ifadeleri şunları içerir:

- Tek bir eşittir işareti kullanılarak yapılan atama ifadesi=
- if Şartlı olarak bir kod bloğunu yürüten ifade, ve else(elif kısaltılmış hali else if) ile birlikte kullanılır.
- Bu ifade, yinelenebilir for bir nesne üzerinde yineleme yaparak her öğeyi, ekli blok tarafından kullanılmak üzere bir değişkene yakalar; döngü bittiğinde değişken silinmez.

- Bu while ifade, mantıksal koşul doğru olduğu sürece bir kod bloğunu yürütür.
- tryEkli kod bloğunda ortaya çıkan istisnaların maddeler (veya Python 3.11'deki istisna grupları için except yeni sözdizimi) tarafından yakalanmasına ve işlenmesine olanak tanıyan ifade ; ^[87] ifade ayrıca , bloğun nasıl çıktığına bakılmaksızın, bloktaki temizleme kodunun her zaman çalıştırılmasını sağlar.except*tryfinally
- raiseBelirtilen bir istisnayı tetiklemek veya yakalanan bir istisnayı yeniden tetiklemek için kullanılan ifade .
- Nesne yönelimli programlamada kullanılmak üzere, class bir kod bloğunu çalıştıran ve yerel ad alanını bir sınıfa bağlayan ifade .
- Bir fonksiyonu veya metodu def tanımlayan ifade .
- with Bir kod bloğunu bir bağlam yöneticisi içine alan, kaynak edinimi başlatmadır (RAII) benzeri davranışa izin veren ve yaygın bir try/finally deyiminin yerini alan ifade ^[88] Bir bağlamın örnekleri arasında, bazı kodlar çalıştırılmadan önce bir kilit edinmek ve ardından kilidi serbest bırakmak; veya bir dosyayı açıp ardından kapatmak yer alır.
- break Döngüden çıkan ifade
- continue Mevcut yinelemenin geri kalanını atlayıp bir sonrakine geçen ifade .
- Bir değişkeni kaldıran ifade del; isimden değere olan referansı siler ve değişken yeniden tanımlanmadan önce ona atıfta bulunulursa hata üretir ^[d]
- Bu pass ifade, sözdizimsel olarak boş bir kod bloğu oluşturmak için gerekli olan ve hiçbir işlem yapmayan (NOP - no operation) bir ifade görevi gören bir komuttur.
- assert Hata ayıklama sırasında geçerli olması gereken koşulları kontrol etmek için kullanılan ifade .
- Üreteç yield fonksiyonundan (ve ayrıca bir operatörden) değer döndüren ifade ; eşzamanlı işlemleri uygulamak için kullanılır.
- return Bir fonksiyondan değer döndürmek için kullanılan ifade .
- Geçerli programda işlevleri veya değişkenleri kullanılabilen modülleri içe aktarmak için kullanılan ve ifadeleri. Python 3.15, yeni bir anahtar kelimeyle tembel içe aktarma için yeni bir işlevsellik ekler: " Anahtar import kelime hem ve ifadeleriyle çalışır ." ^[89] from lazy import from ... import
- " match Ve " ifadeleri, bir ifadeyi bir veya daha fazla durumla karşılaştırarak kontrol akışı önlemi olarak kullanılan, switch ifadesine case benzer bir yapıya sahiptir .

Atama ifadesi () bir adı ayrı, dinamik olarak ayrılmış bir nesneye referans= olarak bağlar . Değişkenler daha sonra herhangi bir zamanda herhangi bir nesneye yeniden bağlanabilir. Python'da, bir değişken adı sabit bir veri türü olmayan genel bir referans tutucudur; ancak her zaman bir türe sahip bir nesneye referans verir . Buna dinamik tiplendirme denir - her değişkenin yalnızca belirli bir türde bir değer içerebildiği statik tipli dillerin aksine .

Python kuyruk çağrısı optimizasyonunu veya birinci sınıf devamlılıkları desteklemez ; Van Rossum'a göre dil asla desteklemeyecektir. ^[90] ^[91] Bununla birlikte, Python'ın jeneratörlerini genişleterek eş zamanlı işlevsellike daha iyi destek sağlanır. ^[92] 2.5'ten önce, jeneratörler tembel yineleyicilerdi ; veriler jeneratörden tek yönlü olarak dışarı aktarılıyordu. Python 2.5'ten itibaren, verileri bir jeneratör fonksiyonuna geri aktarmak mümkündür; ve 3.3 sürümünden itibaren veriler birden fazla yığın seviyesinden geçirilebilir. ^[93]

İfadeler

Python'ın ifadeleri şunları içerir:

- +Matematiksel toplama, çıkarma ve çarpma için kullanılan , - , ve operatörleri *diğer dillere benzer, ancak bölme davranışı farklıdır. Python'da iki tür bölme vardır: tam sayı bölmesi (veya

tam sayı bölmesi) //ve kayan noktalı bölme /. ^[94] Python, üs alma için operatörünü kullanır ******.

- Python, dizeleri birleştirmek için ``concatening`` operatörünü kullanır `+`. Dil, `*` bir dizeyi belirtilen sayıda tekrarlamak için ``concatening`` operatörünü kullanır.
- İnfiks `@` operatörünün, NumPy gibi kütüphaneler tarafından matris çarpımı için kullanılması amaçlanmıştır. ^[95] ^[96]
- Sözdizimi `:=`, `""` Deniz ayısı operatörü, Python 3.8'de tanıtıldı. Bu operatör, daha büyük bir ifadenin parçası olarak değişkenlere değerler atar. ^[97]
- Python'da, `==` iki nesneyi değere göre karşılaştırır. Python'ın ``value`` isoperatörü, nesne kimliklerini karşılaştırmak (yani referansa göre karşılaştırma) için kullanılabilir ve karşılaştırmalar zincirlenebilir; örneğin, ``value` .a <= b <= c`
- Python'da `and`, `or`, ve operatörleri `not Boolean` operatörleri olarak kullanılır.
- Python'da liste anlama adı verilen bir ifade türü ve üretici ifade adı verilen daha genel bir ifade vardır. ^[64]
- Anonim fonksiyonlar lambda ifadeleri kullanılarak uygulanır ; ancak her gövdede yalnızca bir ifade bulunabilir.
- Koşullu ifadeler ^[98] şeklinde yazılır (Bu , birçok diğer dilde yaygın olan operatörden işlenen sırası bakımından farklıdır .) `x if c else yc ? x : y`
- Python, listeler ve demetler arasında bir ayrım yapar . Listeler, ``list`` olarak yazılır , değiştirilebilir ve sözlüklerin anahtarları olarak kullanılamaz (çünkü Python'da sözlük anahtarları değiştirilemez olmalıdır). ``tuple``, ``true`` olarak yazılır , değiştirilemezdir ve bu nedenle, demetin tüm elemanları değiştirilemez olduğu sürece sözlüklerin anahtarları olarak kullanılabilir. ``true`` operatörü, içeriklerini doğrudan değiştirmeden, ancak her ikisinin elemanlarını içeren yeni bir demet üreten iki demeti birleştirmek için kullanılabilir. Örneğin, başlangıçta ``true``'ya eşit olan değişken verildiğinde , ``true`` önce ``true``'yu değerlendirir ve ``true`` sonucunu verir ; bu sonuç daha sonra ``true``'ya geri atanır - böylece demet nesnelerinin değiştirilemez doğasına uyarken ``true``'nın içeriğini etkili bir şekilde "değiştirir" . Parantezler, belirsiz olmayan bağlamlarda demetler için isteğe bağlıdır. ^[99] `[1, 2, 3](1, 2, 3)+t(1, 2, 3)t = t + (4, 5)t + (4, 5)(1, 2, 3, 4, 5)tt`
- Python'da, her biri atanabilir bir şeye (örneğin, bir değişken veya yazılabilir bir özellik) dönüşen birden fazla ifadenin, tıpkı demet değişmezini oluşturmada olduğu gibi ilişkilendirildiği *sıralı açma özelliği bulunur; sonuçlar daha sonra bir atama ifadesinde eşittir işaretinin sol tarafına yerleştirilir. Bu ifade, eşittir işaretinin sağ tarafındaki yinelenebilir nesnenin, sol taraftaki yazılabilir ifadelerle aynı sayıda değer üretmesini bekler; yineleme sırasında, ifade sağda üretilen değerlerin her birini soldaki karşılık gelen ifadeye atar.* ^[100]
- Python'da, C dilindeki format dizelerine %benzer şekilde çalışan bir "dize biçimlendirme" operatörü vardır ; örneğin, şekilde değerlendirilir . Python 2.6+ ve 3+ sürümlerinde , bu operatöre sınıfın yöntemi eklendi , örneğin, . Python 3.6 "f-dizeleri"ni ekledi : ^[101] `printf"spam=%s eggs=%d" % ("blah", 2)"spam=blah eggs=2" format()str"spam={0} eggs={1}".format("blah", 2) spam = "blah"; eggs = 2; f'spam={spam} eggs={eggs}'`
- Python'da dizeler, tamsayılar ve ondalık sayıları toplamak için kullanılan operatörün aynısı kullanılarak "toplanarak" birleştirilebilir ; örneğin, ``include`` değeri ``include`` olur . Dizeler sayı içeriyorsa, tamsayılar yerine dizeler olarak birleştirilirler; örneğin, ``include`` değeri ``include`` olur `."spam" + "eggs""spameggs""2" + "2""22"`
- Python, dize değişmezlerini çeşitli şekillerde destekler :
 - Tek veya çift tırnak işaretleriyle sınırlandırılmıştır; tek ve çift tırnak işaretleri eşdeğer işlevselliğe sahiptir (Unix kabukları , Perl ve Perl'den etkilenen dillerin aksine). Her iki işaret de ters eğik çizgiyi (`)` kaçış karakteri\ olarak kullanır . Dize interpolasyonu, Python 3.6'da "biçimlendirilmiş dize değişmezleri" olarak kullanılabilir hale geldi . ^[101]
 - Üçlü tırnak işaretiyle belirtilmiş, yani üç tek veya çift tırnak işaretiyle başlayıp biten; bu, birden fazla satıra yayılabilir ve shell'lerde, Perl'de ve Ruby'de "here" belgeleri gibi işlev

görebilir .

- Ham dize çeşitleri, dize sabitinin başına `` eklenerek belirtilir . Kaçış dizileri yorumlanmaz; bu nedenle ham dizeler, normal ifadeler ve Windows tarzı yollar gibi sabit ters eğik çizgilerin yaygın olduğu yerlerde kullanışlıdır. (C#@ daki "-quoting" ile karşılaştırın .)
- Python'da listelerde dizi indeksi ve dizi dilimleme `a[key]` ifadeleri bulunur ve bunlar `<dizi indeksi>`, `<dizi dilimleme>` veya `<dizi dilimleme>` şeklinde yazılır . İndeksler sıfır tabanlıdır ve negatif indeksler sona göre belirlenir. Dilimler, başlangıç indeksinden bitiş indeksine kadar (bitiş indeksi dahil değil) elemanları alır. `step` veya `stride` olarak adlandırılan (isteğe bağlı) üçüncü dilim parametresi , elemanların atlanmasına veya tersine çevrilmesine olanak tanır. Dilim indeksleri atlanabilir; örneğin, `<dizi indeksi>` tüm listenin bir kopyasını döndürür. Bir dilimin her elemanı siğ bir kopyadır .`a[start:stop]``a[start:stop:step]``a[:]`

Python'da, Common Lisp , Scheme veya Ruby gibi dillerin aksine, ifadeler ve deyimler arasında katı bir ayırım uygulanır . Bu ayırım, bazı işlevlerin tekrarlanmasına yol açar, örneğin:

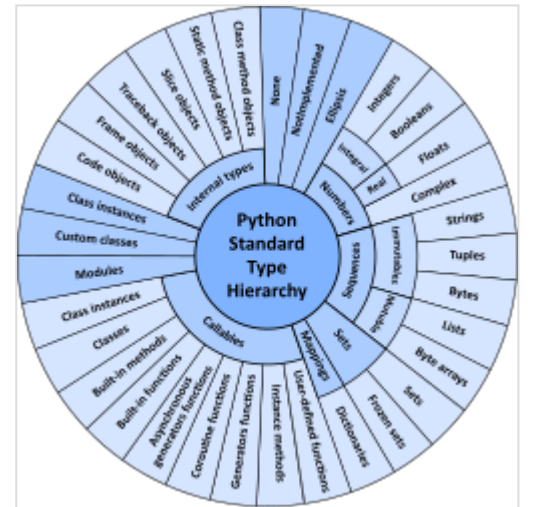
- Liste anlama yapıları `for` ve liste döngüleri arasındaki fark
- Koşullu ifadeler ve if bloklar arasındaki fark
- Python 2'de `function` ile yerleşik fonksiyonlar (`function` bir ifadedir) `eval()` arasındaki fark ; ilki ifadeler için, ikincisi ise `statement`'lar içindir.`exec()` `exec`

Bir ifade, bir cümle öbeğinin parçası olamaz; bu kısıtlama nedeniyle, liste ve liste dictanlama ifadeleri (ve lambda ifadeleri) gibi cümle öbekleri ifade içeremez. Özel bir durum olarak, bir atama ifadesi, koşullu bir ifadenin koşullu ifadesinin parçası olamaz.`a = 1`

Yazma

Python, ördek tipi (duck typing) kullanır ve tipli nesnelere sahipken, tipli olmayan değişken adlarına sahiptir. Tip kısıtlamaları tanımlama zamanında kontrol edilmez; bunun yerine, bir nesne üzerindeki işlemler kullanım zamanında başarısız olabilir ve bu da nesnenin uygun bir tipte olmadığını gösterir. Dinamik olarak tiplendirilmiş olmasına rağmen , Python güçlü bir şekilde tiplendirilmiştir ve kötü tanımlanmış işlemleri (örneğin, bir sayıyı ve bir dizeyi toplamak) yorumlamaya çalışmak yerine yasaklar.

Python, programcılarının sınıflar kullanarak kendi türlerini tanımlamalarına olanak tanır ; bu özellik çoğunlukla nesne yönelimli programlama için kullanılır . Sınıfların yeni örnekleri , örneğin veya gibi sınıf çağrılarak oluşturulur ; sınıflar, metasınıfın (kendisinin bir örneği olan) örnekleridir , bu da metaprogramlama ve yansımaya olanak tanır .`SpamClass()``EggsClass()` type



Python 3'teki standart tür hiyerarşisi

3.0 sürümünden önce Python'da aynı sözdizimini kullanan iki tür sınıf vardı: *eski stil* ve *yeni stil* .
[102] Mevcut Python sürümleri yalnızca yeni stilin semantiğini desteklemektedir.

Python isteğe bağlı tür açıklamalarını destekler . [5] [103] Bu açıklamalar dil tarafından zorunlu kılınmaz, ancak **mypy** gibi harici araçlar tarafından hataları yakalamak için kullanılabilir. Python, typing tür açıklamaları için çeşitli tür adlarını içeren bir modül içerir. [104] [105] Ayrıca, mypy, optimizasyon için tür açıklamalarından yararlanan mypyc adlı bir Python derleyicisini destekler. [106]

Python 3'ün yerleşik veri tiplerinin özeti

Tip	Değişkenlik	Tanım	Sözdizimi örnekleri
bool	değişmez	<u>Mantıksal değer</u>	True False
bytearray	değiştirilebilir	<u>Bayt dizisi</u>	<code>bytearray(b'Some ASCII')</code> <code>bytearray(b"Some ASCII")</code> <code>bytearray([119, 105, 107, 105])</code>
bytes	değişmez	Bayt dizisi	<code>b'Some ASCII'</code> <code>b"Some ASCII"</code> <code>bytes([119, 105, 107, 105])</code>
complex	değişmez	Gerçek ve sanal kısımları olan <u>karmaşık sayı</u>	<code>3+2.7j</code> <code>3 + 2.7j</code> <code>5j</code>
dict	değiştirilebilir	Anahtar ve değer çiftlerinden oluşan <u>ilişkisel dizi</u> (veya sözlük); karışık türler (anahtarlar ve değerler) içerebilir; anahtarlar hashlenebilir bir tür olmalıdır.	<code>{'key1': 1.0, 3: False}</code> <code>{}</code>
types.EllipsisType	değişmez	<u>NumPy</u> dizilerinde indeks olarak kullanılacak üç nokta (ellipsis) yer tutucusu.	<code>...</code> Ellipsis
float	değişmez	64 bit çift hassasiyetli kayan noktalı sayı! (CPython 3.11'den beri desteklenen tüm sürümlerde 53 bit hassasiyetli IEEE 754 sayısı; ^[107] ayrıca pratikte 3.10 ve daha eski sürümlerde de mevcuttur, ancak teknik olarak hassasiyet makineye bağlıdır ^[108]) . Python'ın yerleşik türü hem 64 bit çift hassasiyetli hem de 32 bit kayan noktalı sayıları destekler ve Python'ın standart kütüphane modülü ayrıca 16 bit yarım kayan noktalı sayıları da destekler. NumPy ve Pandas gibi Python paketleri de 32 bit veya daha fazla kayan noktalı sayıları destekleyebilir, örneğin; ancak yarım kayan noktalı sayılar için destek çoğu pakette genellikle eksik veya mevcut değildir. ^[109] 16 bit <code>bfloat</code> birkaç pakette desteklenmektedir. ^[110] <code>memoryviewstructnumpy.half</code> Çoğu Python uygulaması çift duyarlılık sayı türünü desteklemeyi tercih eder, ancak <u>gömülü programlama</u> için <u>MicroPython</u> gibi bazı sıra dışı (alt küme) uygulamalar varsayılan kayan nokta sayısı olarak 32 bit IEEE kayan nokta sayısını destekler.	1.33333

		Kullanıcılar, MICROPY_FLOAT_IMPL'yi MICROPY_FLOAT_IMPL_DOUBLE olarak ayarlayarak 64 bit'i tercih edebilirler.	
frozenset	değişmez	Sıralı olmayan <u>küme</u> , yinelenen eleman içermez; eğer hashlenebilir ise karışık türler içerebilir.	<code>frozenset({4.0, 'string', True})</code> <code>frozenset()</code>
int	değişmez	Sınırsız büyüklükte <u>tamsayı</u> ^[111] (yani makine tamsayıları kullanılmıyor; örneğin Python paketi NumPy, hızlandırma için sabit boyutlu tamsayılar kullanır (ve farklı boyutlara izin verir), bu türler numpy.byteve gibi numpy.ulonglong, tüm bu türlerin etrafında dönme olasılığı vardır, ancak sayı ne kadar büyükse olasılık o kadar düşüktür)	42
list	değiştirilebilir	<u>Liste</u> , farklı türlerde içerik içerebilir.	<code>[4.0, 'string', True]</code> <code>[]</code>
types.NoneType	değişmez	Bir değerin yokluğunu temsil eden nesne, diğer dillerde genellikle <u>null</u> olarak adlandırılır.	None
types.NotImplementedType	değişmez	<u>Aşırı yüklenmiş operatörlerden</u> , desteklenmeyen işlenen türlerini belirtmek için döndürülebilen bir yer tutucu.	NotImplemented
range	değişmez	Döngülerde belirli sayıda kez yineleme için yaygın olarak kullanılan, <u>değiştirilemez</u> bir sayı dizisi ^[112] for	<code>range(-1, 10)</code> <code>range(10, -5, -2)</code>
set	değiştirilebilir	Sıralı olmayan <u>küme</u> , yinelenen eleman içermez; eğer hashlenebilir ise karışık türler içerebilir.	<code>{4.0, 'string', True}</code> <code>set()</code>
str	değişmez	Karakter <u>dizisi</u> : Unicode kod noktalarının dizisi	<code>'Wikipedia'</code> <code>"Wikipedia"</code> <code>"</code> <i>Birden fazla satırı kapsayan</i> <code>"</code>
tuple	değişmez	<u>Tuple</u> , farklı veri tiplerini içerebilir.	<code>(4.0, 'string', True)</code> <code>('single element',)</code> <code>()</code>

Aritmetik işlemler

Python, aritmetik operatörler (+, -, *, /), taban bölme operatörü //ve modül operatörü % için geleneksel semboller içerir . (Modül operatöründe kalan negatif olabilir, örneğin .) Python ayrıca üs alma için sembolünü , örneğin ve , ve matris çarpma operatörünü de sunar .^[113] Bu operatörler geleneksel matematikteki gibi çalışır; aynı öncelik kurallarıyla , infix operatörler ve sırasıyla pozitif ve negatif sayıları temsil etmek için tekli de olabilir . $4 \% -3 == -2**5**3 == 1259**0.5 == 3.0@+-$

Tamsayılar arasındaki bölme, kayan noktalı sonuçlar üretir. Bölmenin davranışı zaman içinde önemli ölçüde değişti: ^[114]

- Python'ın mevcut sürümü (yani 3.0'dan beri), /operatörü her zaman kayan noktalı bölmeyi temsil edecek şekilde değiştirdi, örneğin $.5/2 == 2.5$
- Taban bölme //operatörü tanıtıldı, yani $7//3 == 2$, $-7//3 == -3$, $7.5//3 == 2.0$, ve $-7.5//3 == -3.0$. Python 2.7 için, ifadeyi eklemek, Python 2.7'deki bir modülün Python 3.x bölme kurallarını kullanmasına olanak tanır (yukarıya bakın).`from __future__ import division`

Python terimleriyle, operatör *gerçek bölmeyi* (veya basitçe *bölmeyi*) /temsil ederken , operatör *taban bölmeyi* temsil eder . 3.0 sürümünden önce, operatör *klasik bölmeyi* temsil eder .^[114]///

Rounding towards negative infinity, though a different method than in most languages, adds consistency to Python. For instance, this rounding implies that the equation $(a + b)//b == a//b + 1$ is always true. Also, the rounding implies that the equation $b*(a//b) + a\%b == a$ is valid for both positive and negative values of a . As expected, the result of $a\%b$ lies in the half-open interval $[0, b)$, where b is a positive integer; however, maintaining the validity of the equation requires that the result must lie in the interval $(b, 0]$ when b is negative.^[115]

Python provides a round function for rounding a float to the nearest integer. For tie-breaking, Python 3 uses the *round to even* method: `round(1.5)` and `round(2.5)` both produce 2.^[116] Python versions before 3 used the round-away-from-zero method: `round(0.5)` is 1.0, and `round(-0.5)` is -1.0.^[117]

Python allows Boolean expressions that contain multiple equality relations to be consistent with general usage in mathematics. For example, the expression $a < b < c$ tests whether a is less than b and b is less than c .^[118] C-derived languages interpret this expression differently: in C, the expression would first evaluate $a < b$, resulting in 0 or 1, and that result would then be compared with c .^[119]

Python uses arbitrary-precision arithmetic for all integer operations. The `Decimal` type/class in the `decimal` module provides decimal floating-point numbers to a pre-defined arbitrary precision with several rounding modes.^[120] The `Fraction` class in the `fractions` module provides arbitrary precision for rational numbers.^[121]

Due to Python's extensive mathematics library and the third-party library NumPy, the language is frequently used for scientific scripting in tasks such as numerical data processing and manipulation.^{[122][123]}

Function syntax

Functions are created in Python by using the `def` keyword. A function is defined similarly to how it is called, by first providing the function name and then the required parameters. Here is an example of a function that prints its inputs:

```
def printer(input1, input2 = "already there"):
    print(input1)
    print(input2)

printer("hello")

# Example output:
# hello
# already there
```

To assign a default value to a function parameter in case no actual value is provided at run time, variable-definition syntax can be used inside the function header.

Code examples

"Hello, World!" program:

```
print('Hello, World!')
```

Program to calculate the factorial of a non-negative integer:

```
1 text = input('Type a number, and its factorial will be printed: ')
2 n = int(text)
3
4 if n < 0:
5     raise ValueError('You must enter a non-negative integer')
6
7 factorial = 1
8 for i in range(2, n + 1):
9     factorial *= i
10
11 print(factorial)
```

Libraries

Python's large standard library^[124] is commonly cited as one of its greatest strengths. For Internet-facing applications, many standard formats and protocols such as MIME and HTTP are supported. The language includes modules for creating graphical user interfaces, connecting to relational databases, generating pseudorandom numbers, arithmetic with arbitrary-precision decimals,^[120] manipulating regular expressions, and unit testing.

Some parts of the standard library are covered by specifications—for example, the Web Server Gateway Interface (WSGI) implementation `wsgiref` follows PEP 333^[125]—but most parts are specified by their code, internal documentation, and test suites. However, because most of the standard library is cross-platform Python code, only a few modules must be altered or rewritten for variant implementations.

As of 13 March 2025, the Python Package Index (PyPI), the official repository for third-party Python software, contains over 614,339^[126] packages.

Development environments

Most Python implementations (including CPython) include a read–eval–print loop (REPL); this permits the environment to function as a command line interpreter, with which users enter statements sequentially and receive results immediately.^[127]

Also, CPython is bundled with an integrated development environment (IDE) called IDLE,^[128] which is oriented toward beginners.

Other shells, including IDLE and IPython, add additional capabilities such as improved auto-completion, session-state retention, and syntax highlighting.^{[128][129]}

Standard desktop IDEs include PyCharm, Spyder, and Visual Studio Code;^[130] there are web browser-based IDEs, such as the following environments:

- Jupyter Notebooks, an open-source interactive computing platform;^[131]
- PythonAnywhere, a browser-based IDE and hosting environment; and
- Canopy, a commercial IDE from Enthought that emphasizes scientific computing.^{[132][133]}

Implementations

Reference implementation

CPython is the reference implementation of Python. This implementation is written in C; meeting the C11 standard^[134] since version 3.11. Older versions use the C89 standard with several select C99 features, but third-party extensions are not limited to older C versions—e.g., they can be implemented using C11 or C++.^{[135][136]} CPython compiles Python programs into an intermediate bytecode,^[137] which is then executed by a virtual machine.^[138] CPython is distributed with a large standard library written in a combination of C and native Python.

CPython is available for many platforms, including Windows and most modern Unix-like systems, including macOS (and Apple M1 Macs, since Python 3.9.1, using an experimental installer). Starting with Python 3.9, the Python installer intentionally fails to install on Windows 7 and 8;^{[139][140]} Windows XP was supported until Python 3.5. Old Python versions unofficially support VMS (mostly supporting^[141]) and OpenVMS x86-64 has Python 3.10 support.^{[142][143]} Platform portability was one of Python's earliest priorities.^[144] During development of Python 1 and 2, even OS/2 and Solaris were supported;^[8] since that time, support has been dropped for many platforms.

All current Python versions (since 3.7) support only operating systems that feature multithreading, by now supporting not nearly as many operating systems (dropping many outdated) than in the past.

Limitations of the reference implementation

- The energy usage of Python with CPython for typically written code is much worse than C by a factor of 75.88.^[145]
- The throughput of Python with CPython for typically written code is worse than C by a factor of 71.9.^[145]
- The average memory usage of CPython for typically written code is worse than C by a factor of 2.4.^[145]

Other implementations

All alternative implementations have at least slightly different semantics. For example, an alternative may include unordered dictionaries, in contrast to other current Python versions. As another example in the larger Python ecosystem, PyPy does not support the full C Python API.

Creating an executable with Python often is done by bundling an entire Python interpreter into the executable, which causes binary sizes to be massive for small programs,^[146] yet there exist implementations that are capable of truly compiling Python. Alternative implementations include the following:

- PyPy is a faster, compliant interpreter of Python 2.7 and 3.11.^{[147][148]} PyPy's just-in-time compiler often improves speed significantly relative to CPython, but PyPy does not support some libraries written in C.^[149] PyPy offers support for the RISC-V instruction-set architecture.
- Codon is an implementation with an ahead-of-time (AOT) compiler, which compiles a statically-typed Python-like language whose "syntax and semantics are nearly identical to Python's, there are some notable differences"^[150] For example, Codon uses 64-bit machine integers for speed, not arbitrarily as with Python; Codon developers claim that speedups over CPython are usually on the order of ten to a hundred times. Codon compiles to machine code (via LLVM) and supports native multithreading.^[151] Codon can also compile to Python extension modules that can be imported and used from Python.
- MicroPython and CircuitPython are Python 3 variants that are optimized for microcontrollers, including the Lego Mindstorms EV3.^[152]
- Pyston is a variant of the Python runtime that uses just-in-time compilation to speed up execution of Python programs.^[153]
- Cinder is a performance-oriented fork of CPython 3.8 that features a number of optimizations, including bytecode inline caching, eager evaluation of coroutines, a method-at-a-time JIT, and an experimental bytecode compiler.^[154]
- The Snek^{[155][156][157]} embedded computing language "is Python-inspired, but it is not Python. It is possible to write Snek programs that run under a full Python system, but most Python programs will not run under Snek."^[158] Snek is compatible with 8-bit AVR microcontrollers such as ATmega 328P-based Arduino, as well as larger microcontrollers that are compatible with MicroPython. Snek is an imperative language that (unlike Python) omits object-oriented programming. Snek supports only one numeric data type, which features 32-bit single precision (resembling JavaScript numbers, though smaller).
- RustPython is an implementation written in Rust language. It aims to be compatible with CPython, including its C-ABI.^[159] Currently, it is used in GrepTimeDB (<https://www.greptime.com/product/db>) and Ruff (<https://astral.sh/ruff>) among other projects.

Unsupported implementations

Stackless Python is a significant fork of CPython that implements microthreads. This implementation uses the call stack differently, thus allowing massively concurrent programs. PyPy also offers a stackless version.^[160]

Just-in-time Python compilers have been developed, but are now unsupported:

- Google began a project named Unladen Swallow in 2009: this project aimed to speed up the Python interpreter five-fold by using LLVM, and improve multithreading capability for scaling to thousands of cores,^[161] while typical implementations are limited by the global interpreter lock.
- Psyco is a discontinued just-in-time specializing compiler, which integrates with CPython and transforms bytecode to machine code at runtime. The emitted code is specialized for certain data types and is faster than standard Python code. Psyco does not support Python 2.7 or later.
- PyS60 was a Python 2 interpreter for Series 60 mobile phones, which was released by Nokia in 2005. The interpreter implemented many modules from Python's standard library, as well as additional modules for integration with the Symbian operating system. The Nokia N900 also supports Python through the GTK widget library, allowing programs to be written and run on the target device.^[162]

Transpilers to other languages

There are several compilers/transpilers to high-level object languages; the source language is unrestricted Python, a subset of Python, or a language similar to Python:

- Brython^[163] and Transcrypt^{[164][165]} compile Python to JavaScript.
- Cython compiles a superset of Python to C. The resulting code can be used with Python via direct C-level API calls into the Python interpreter.
- PyJL compiles/transpiles a subset of Python to "human-readable, maintainable, and high-performance Julia source code".^[75] Despite the developers' performance claims, this is not possible for *arbitrary* Python code; that is, compiling to a faster language or machine code is known to be impossible in the general case. The semantics of Python might potentially be changed, but in many cases speedup is possible with few or no changes in the Python code. The faster Julia source code can then be used from Python or compiled to machine code.
- Nuitka compiles Python into C.^[166] This compiler works with Python 3.4 to 3.13 (and 2.6 and 2.7) for Python's main supported platforms (and Windows 7 or even Windows XP) and for Android. The compiler developers claim full support for Python 3.10, partial support for Python 3.11 and 3.12, and experimental support for Python 3.13. Nuitka supports macOS including Apple Silicon-based versions. The compiler is free of cost, though it has commercial add-ons (e.g., for hiding source code).
- Numba is a JIT compiler that is used from Python; the compiler translates a subset of Python and NumPy code into fast machine code. This tool is enabled by adding a decorator to the relevant Python code.
- Pythran compiles a subset of Python 3 to C++ (C++11).^[167]
- RPython can be compiled to C, and it is used to build the PyPy interpreter for Python.
- The Python → 11l → C++ transpiler^[168] compiles a subset of Python 3 to C++ (C++17).

There are also specialized compilers:

- MyHDL is a Python-based hardware description language (HDL) that converts MyHDL code to Verilog or VHDL code.

Some older projects existed, as well as compilers not designed for use with Python 3.x and related syntax:

- Google's [Grumpy](#) [transpiles](#) Python 2 to [Go](#).^{[169][170][171]} The latest release was in 2017.
- [IronPython](#) allows running Python 2.7 programs with the [.NET Common Language Runtime](#).^[172] An alpha version (released in 2021), is available for "Python 3.4, although features and behaviors from later versions may be included."^[173]
- [Jython](#) compiles Python 2.7 to Java bytecode, allowing the use of Java libraries from a Python program.^[174]
- [Pyrex](#) (last released in 2010) and [Shed Skin](#) (last released in 2013) compile to C and C++ respectively.

Performance

A performance comparison among various Python implementations, using a non-numerical (combinatorial) workload, was presented at EuroSciPy '13.^[175] In addition, Python's performance relative to other programming languages is benchmarked by [The Computer Language Benchmarks Game](#).^[176]

There are several approaches to optimizing Python performance, despite the inherent slowness of an [interpreted language](#). These approaches include the following strategies or tools:

- [Just-in-time compilation](#): Dynamically compiling parts of a Python program during the execution of the program. This technique is used in libraries such as [Numba](#) and [PyPy](#).
- [Static compilation](#): Sometimes, Python code can be compiled into machine code sometime before execution. An example of this approach is [Cython](#), which compiles Python into C.
- [Concurrency and parallelism](#): Multiple tasks can be run simultaneously. Python contains modules such as ``multiprocessing`` to support this form of parallelism. Moreover, this approach helps to overcome limitations of the [Global Interpreter Lock](#) (GIL) in CPU tasks.
- [Efficient data structures](#): Performance can also be improved by using data types such as [Set](#) for membership tests, or [deque](#) from [collections](#) for [queue](#) operations.
- Performance gains can be observed by utilizing libraries such as [NumPy](#). Most high performance Python libraries use [C](#) or [Fortran](#) under the hood instead of the Python interpreter.^[177]

Language development

Python's development is conducted mostly through the *Python Enhancement Proposal* (PEP) process; this process is the primary mechanism for proposing major new features, collecting community input on issues, and documenting Python design decisions.^[178] Python coding style is covered in PEP 8.^[86] Outstanding PEPs are reviewed and commented on by the Python community and the steering council.^[178]

Enhancement of the language corresponds with development of the CPython reference implementation. The mailing list python-dev is the primary forum for the language's development. Specific issues were originally discussed in the [Roundup bug tracker](#) hosted by the foundation.^[179] In 2022, all issues and discussions were migrated to [GitHub](#).^[180] Development originally took place on a [self-hosted](#) source-code repository running [Mercurial](#), until Python moved to GitHub in January 2017.^[181]

CPython's public releases have three types, distinguished by which part of the version number is incremented:

- *Backward-incompatible versions*, where code is expected to break and must be manually ported. The first part of the version number is incremented. These releases happen infrequently—version 3.0 was released 8 years after 2.0. According to Guido van Rossum, a version 4.0 will probably never exist.^[182]
- *Major or "feature" releases* are largely compatible with the previous version but introduce new features. The second part of the version number is incremented. Starting with Python 3.9, these releases are expected to occur annually.^{[183][184]} Each major version is supported by bug fixes for several years after its release.^[185]
- *Bug fix releases*,^[186] which introduce no new features, occur approximately every three months; these releases are made when a sufficient number of bugs have been fixed upstream since the last release. Security vulnerabilities are also patched in these releases. The third and final part of the version number is incremented.^[186]

Many alpha, beta, and release-candidates are also released as previews and for testing before final releases. Although there is a rough schedule for releases, they are often delayed if the code is not ready yet. Python's development team monitors the state of the code by running a large unit test suite during development.^[187]

The major academic conference on Python is PyCon. Also, there are special Python mentoring programs, such as PyLadies.

Naming

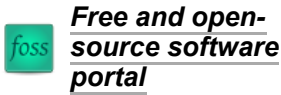
Python's name is inspired by the British comedy group Monty Python, whom Python creator Guido van Rossum enjoyed while developing the language. Monty Python references appear frequently in Python code and culture;^[188] for example, the metasyntactic variables often used in Python literature are *spam* and *eggs*, rather than the traditional *foo* and *bar*.^{[188][189]} Also, the official Python documentation contains various references to Monty Python routines.^{[190][191]} Python users are sometimes referred to as "Pythonistas".^[192]

Languages influenced by Python

- Cobra has an *Acknowledgements* document that lists Python first among influencing languages.^[193]
- ECMAScript and JavaScript borrowed iterators and generators from Python.^[194]
- Go is designed for "speed of working in a dynamic language like Python".^[195]
- Julia was designed to be "as usable for general programming as Python".^[196]
- Mojo is almost^{[34][197]} a superset of Python.^[198]
- GDScript is strongly influenced by Python.^[199]
- Groovy, Boo, CoffeeScript, F#, Nim, Ruby,^[35] Swift,^[36] and V^[37] have been influenced, as well.

See also





- [List of machine learning and deep learning software for Python](#)
- [List of Python programming books](#)
- [pip \(package manager\)](#)
- [Pydoc](#)
- [NumPy](#)
- [SciPy](#)
- [Jupyter](#)
- [PyTorch](#)
- [Cython](#)
- [CPython](#)
- [Mojo](#)
- [Pygame](#)
- [PyQt](#)
- [PyGTK](#)
- [PyPy](#)
- [PyCon](#)
- [Google Colab](#) – zero setup [online IDE](#) that runs Python
- [Ren'Py](#)

Notes

- a. since 3.5, but those hints are ignored, except with unofficial tools^[5]
 - b.
 - **Tier 1:** 64-bit [Linux](#), [macOS](#); 64- and 32-bit [Windows](#)^[6]
 - **Tier 2:** E.g. 32-bit [WebAssembly](#) (WASI)
 - **Tier 3:** 64-bit [Android](#),^[7] [iOS](#), [FreeBSD](#), and (32-bit) [Raspberry Pi OS](#)
Unofficial (or has been known to work): Other [Unix-like/BSD](#) variants) and a few other platforms^{[8][9][10]}
 - c. He has since come out of retirement and is self-titled "BDFL-emeritus".
 - d. `del` in Python does not behave the same way `delete` in languages such as [C++](#) does, where such a word is used to call the [destructor](#) and deallocate heap memory.
1. Also known as **PEP**^[80]
 2. to describe a new feature on Python
 3. to describe a design issue or providing a general guideline
 4. to describe a process surrounding in Python

References

1. "General Python FAQ – Python 3 documentation" (<https://docs.python.org/3/faq/general.html#what-is-python>). *docs.python.org*. Retrieved 7 July 2024.
2. "Python 0.9.1 part 01/21" (<https://www.tuhs.org/Usenet/alt.sources/1991-February/001749.html>). alt.sources archives. Archived (<https://web.archive.org/web/20210811171015/https://www.tuhs.org/Usenet/alt.sources/1991-February/001749.html>) from the original on 11 August 2021. Retrieved 11 August 2021.

3. "Python 3.14.6 and 3.13.14 are now available!" (<https://blog.python.org/2026/06/python-3146-31314/>). 10 June 2026. Retrieved 11 June 2026.
4. "Why is Python a dynamic language and also a strongly typed language" (<https://wiki.python.org/moin/Why%20is%20Python%20a%20dynamic%20language%20and%20also%20a%20strongly%20typed%20language>). *Python Wiki*. Archived (<https://web.archive.org/web/20210314173706/https://wiki.python.org/moin/Why%20is%20Python%20a%20dynamic%20language%20and%20also%20a%20strongly%20typed%20language>) from the original on 14 March 2021. Retrieved 27 January 2021.
5. van Rossum, Guido; Levkivskyi, Ivan. "PEP 483 – The Theory of Type Hints" (<https://www.python.org/dev/peps/pep-0483/>). *Python Enhancement Proposals (PEPs)*. Archived (<https://web.archive.org/web/20200614153558/https://www.python.org/dev/peps/pep-0483/>) from the original on 14 June 2020. Retrieved 14 June 2018.
6. von Löwis, Martin; Cannon, Brett. "PEP 11 – CPython platform support" (<https://peps.python.org/pep-0011/>). *Python Enhancement Proposals (PEPs)*. Retrieved 22 April 2024.
7. "PEP 738 – Adding Android as a supported platform | peps.python.org" (<https://peps.python.org/pep-0738/>). *Python Enhancement Proposals (PEPs)*. Retrieved 19 May 2024.
8. "Download Python for Other Platforms" (<https://www.python.org/download/other/>). *Python.org*. Archived (<https://web.archive.org/web/20201127015815/https://www.python.org/download/other/>) from the original on 27 November 2020. Retrieved 18 August 2023.
9. "test – Regression tests package for Python" (https://docs.python.org/3.7/library/test.html?highlight=android#test.support.is_android). *Python 3.7.17 documentation*. Archived (https://web.archive.org/web/20220517151240/https://docs.python.org/3.7/library/test.html?highlight=android#test.support.is_android) from the original on 17 May 2022. Retrieved 17 May 2022.
10. "platform – Access to underlying platform's identifying data" (<https://docs.python.org/3.10/library/platform.html?highlight=android>). *Python 3.10.4 documentation*. Archived (<https://web.archive.org/web/20220517150826/https://docs.python.org/3.10/library/platform.html?highlight=android>) from the original on 17 May 2022. Retrieved 17 May 2022.
11. Venners, Bill (13 January 2003). "The Making of Python" (<http://www.artima.com/intv/pythonP.html>). *Artima Developer*. Artima. Archived (<https://web.archive.org/web/20160901183332/http://www.artima.com/intv/pythonP.html>) from the original on 1 September 2016. Retrieved 22 March 2007.
12. Cannon, Brett (20 February 2015). "PEP 488 – Elimination of PYO files" (<https://peps.python.org/pep-0488/>). *Python Enhancement Proposals (PEPs)*. Archived (<https://web.archive.org/web/20260116004856/https://peps.python.org/pep-0488/>) from the original on 16 January 2026. Retrieved 28 February 2026. "A PYC file is the bytecode file generated and read from when no optimization level is specified at interpreter startup [...] .pyc"
13. Ahlstrom, James C. (11 October 2001). "PEP 273 – Import Modules from Zip Archives" (<https://peps.python.org/pep-0273/>). *Python Enhancement Proposals (PEPs)*. Archived (<https://web.archive.org/web/20260225044635/https://peps.python.org/pep-0273/>) from the original on 25 February 2026. Retrieved 28 February 2026. "Dynamic modules have extensions like .dll, .pyd, and .so."
14. Harper Smith, Emma (9 September 2017). "PEP 561 – Distributing and Packaging Type Information" (<https://peps.python.org/pep-0561/>). *Python Enhancement Proposals (PEPs)*. Archived (<https://web.archive.org/web/20251207013328/https://peps.python.org/pep-0561/>) from the original on 7 December 2025. Retrieved 28 February 2026. "'stubs' - files containing only type information, empty of runtime code (the filename ends in .pyi)."
15. Hammond, Mark; von Löwis, Martin (15 March 2011). "PEP 397 – Python launcher for Windows" (<https://peps.python.org/pep-0397/>). *Python Enhancement Proposals (PEPs)*. Archived (<https://web.archive.org/web/20260204012409/https://peps.python.org/pep-0397/>) from the original on 4 February 2026. Retrieved 28 February 2026. "[...] the 'console' version of the launcher is associated with .py files and the 'windows' version associated with .pyw files."

16. Holth, Daniel; Moore, Paul (30 March 2013). "PEP 0441 – Improving Python ZIP Application Support" (<https://www.python.org/dev/peps/pep-0441/>). *Python Enhancement Proposals (PEPs)*. Archived (<https://web.archive.org/web/20151116044812/https://www.python.org/dev/peps/pep-0441/>) from the original on 16 November 2015. Retrieved 12 November 2015.
17. "Starlark Language" (<https://docs.bazel.build/versions/master/skylark/language.html>). *bazel.build*. Archived (<https://web.archive.org/web/20200615140534/https://docs.bazel.build/versions/master/skylark/language.html>) from the original on 15 June 2020. Retrieved 25 May 2019.
18. "Why was Python created in the first place?" (<https://docs.python.org/faq/general.html#why-was-python-created-in-the-first-place>). *General Python FAQ*. Python Software Foundation. Archived (<https://web.archive.org/web/20121024164224/http://docs.python.org/faq/general.html#why-was-python-created-in-the-first-place>) from the original on 24 October 2012. Retrieved 22 March 2007. "I had extensive experience with implementing an interpreted language in the ABC group at CWI, and from working with this group I had learned a lot about language design. This is the origin of many Python features, including the use of indentation for statement grouping and the inclusion of very high-level data types (although the details are all different in Python)."
19. "Ada 83 Reference Manual (raise statement)" (<https://archive.adaic.com/standards/83lrm/html/lrm-11-03.html#11.3>). *archive.adaic.com*. Archived (<https://web.archive.org/web/20191022155758/http://archive.adaic.com/standards/83lrm/html/lrm-11-03.html#11.3>) from the original on 22 October 2019. Retrieved 7 January 2020.
20. Kuchling, Andrew M. (22 December 2006). "Interview with Guido van Rossum (July 1998)" (<https://web.archive.org/web/20070501105422/http://www.amk.ca/python/writing/gvr-interview>). *amk.ca*. Archived from the original (<http://www.amk.ca/python/writing/gvr-interview>) on 1 May 2007. Retrieved 12 March 2012. "I'd spent a summer at DEC's Systems Research Center, which introduced me to Modula-2+; the Modula-3 final report was being written there at about the same time. What I learned there later showed up in Python's exception handling, modules, and the fact that methods explicitly contain 'self' in their parameter list. String slicing came from Algol-68 and Icon."
21. "itertools – Functions creating iterators for efficient looping" (<https://docs.python.org/3.7/library/itertools.html>). *Python 3.7.17 documentation*. Archived (<https://web.archive.org/web/20200614153629/https://docs.python.org/3/library/itertools.html>) from the original on 14 June 2020. Retrieved 22 November 2016. "This module implements a number of iterator building blocks inspired by constructs from APL, Haskell, and SML."
22. van Rossum, Guido (1993). "An Introduction to Python for UNIX/C Programmers". *Proceedings of the NLUUG Najaarsconferentie (Dutch UNIX Users Group)*. CiteSeerX 10.1.1.38.2023 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.38.2023>). "even though the design of C is far from ideal, its influence on Python is considerable."
23. "Classes" (<https://docs.python.org/tutorial/classes.html>). *The Python Tutorial*. Python Software Foundation. Archived (<https://web.archive.org/web/20121023030209/http://docs.python.org/tutorial/classes.html>) from the original on 23 October 2012. Retrieved 20 February 2012. "It is a mixture of the class mechanisms found in C++ and Modula-3"
24. Lundh, Fredrik. "Call By Object" (<http://effbot.org/zone/call-by-object.htm>). *effbot.org*. Archived (<https://web.archive.org/web/20191123043655/http://effbot.org/zone/call-by-object.htm>) from the original on 23 November 2019. Retrieved 21 November 2017. "replace "CLU" with "Python", "record" with "instance", and "procedure" with "function or method", and you get a pretty accurate description of Python's object model."
25. Simionato, Michele. "The Python 2.3 Method Resolution Order" (<https://www.python.org/download/releases/2.3/mro/>). Python Software Foundation. Archived (<https://web.archive.org/web/20200820231854/https://www.python.org/download/releases/2.3/mro/>) from the original on 20 August 2020. Retrieved 29 July 2014. "The C3 method itself has nothing to do with Python, since it was invented by people working on Dylan and it is described in a paper intended for lispers"

26. Kuchling, A. M. "Functional Programming HOWTO" (<https://docs.python.org/howto/functional.html>). *Python v2.7.2 documentation*. Python Software Foundation. Archived (<https://web.archive.org/web/20121024163217/http://docs.python.org/howto/functional.html>) from the original on 24 October 2012. Retrieved 9 February 2012. "List comprehensions and generator expressions [...] are a concise notation for such operations, borrowed from the functional programming language Haskell."
27. Schemenauer, Neil; Peters, Tim; Hetland, Magnus Lie (18 May 2001). "PEP 255 – Simple Generators" (<https://www.python.org/dev/peps/pep-0255/>). *Python Enhancement Proposals*. Python Software Foundation. Archived (<https://web.archive.org/web/20200605012926/https://www.python.org/dev/peps/pep-0255/>) from the original on 5 June 2020. Retrieved 9 February 2012.
28. "More Control Flow Tools" (<https://docs.python.org/3.2/tutorial/controlflow.html>). *Python 3 documentation*. Python Software Foundation. Archived (<https://web.archive.org/web/20160604080843/https://docs.python.org/3.2/tutorial/controlflow.html>) from the original on 4 June 2016. Retrieved 24 July 2015. "By popular demand, a few features commonly found in functional programming languages like Lisp have been added to Python. With the lambda keyword, small anonymous functions can be created."
29. "re – Regular expression operations" (<https://docs.python.org/3.10/library/re.html>). *Python 3.10.6 documentation*. Archived (<https://web.archive.org/web/20180718132241/https://docs.python.org/3/library/re.html>) from the original on 18 July 2018. Retrieved 6 September 2022. "This module provides regular expression matching operations similar to those found in Perl."
30. "CoffeeScript" (<https://coffeescript.org/>). *coffeescript.org*. Archived (<https://web.archive.org/web/20200612100004/http://coffeescript.org/>) from the original on 12 June 2020. Retrieved 3 July 2018.
31. Rauschmayer, Axel (24 February 2013). "Perl and Python influences in JavaScript" (<https://www.2ality.com/2013/02/javascript-influences.html>). *2ality.com*. Archived (<https://web.archive.org/web/20181226141121/http://2ality.com/2013/02/javascript-influences.html%0A>) from the original on 26 December 2018. Retrieved 15 May 2015.
32. Rauschmayer, Axel. "Chapter 3: The Nature of JavaScript; Influences" (<https://speakingjs.com/es5/ch03.html>). *Speaking JavaScript*. O'Reilly. Archived (<https://web.archive.org/web/20181226141123/http://speakingjs.com/es5/ch03.html%0A>) from the original on 26 December 2018. Retrieved 15 May 2015.
33. Bezanson, Jeff; Karpinski, Stefan; Shah, Viral B.; Edelman, Alan (February 2012). "Why We Created Julia" (<https://julialang.org/blog/2012/02/why-we-created-julia>). *Julia website*. Archived (<https://web.archive.org/web/20200502144010/https://julialang.org/blog/2012/02/why-we-created-julia/>) from the original on 2 May 2020. Retrieved 5 June 2014. "We want something as usable for general programming as Python [...]"
34. Krill, Paul (4 May 2023). "Mojo language marries Python and MLIR for AI development" (<https://www.infoworld.com/article/3695588/mojo-language-marries-python-and-mlir-for-ai-development.html>). *InfoWorld*. Archived (<https://web.archive.org/web/20230505064554/https://www.infoworld.com/article/3695588/mojo-language-marries-python-and-mlir-for-ai-development.html>) from the original on 5 May 2023. Retrieved 5 May 2023.
35. Bini, Ola (2007). *Practical JRuby on Rails Web 2.0 Projects: bringing Ruby on Rails to the Java platform* (<https://archive.org/details/practicaljrubbyon0000bini/page/3>). Berkeley: APress. p. 3 (<https://archive.org/details/practicaljrubbyon0000bini/page/3>). ISBN 978-1-59059-881-8.
36. Lattner, Chris (3 June 2014). "Chris Lattner's Homepage" (<http://nondot.org/sabre/>). Chris Lattner. Archived (<https://web.archive.org/web/20181225175312/http://nondot.org/sabre/>) from the original on 25 December 2018. Retrieved 3 June 2014. "The Swift language is the product of tireless effort from a team of language experts, documentation gurus, compiler optimization ninjas, and an incredibly important internal dogfooding group who provided feedback to help refine and battle-test ideas. Of course, it also greatly benefited from the experiences hard-won by many other languages in the field, drawing ideas from Objective-C, Rust, Haskell, Ruby, Python, C#, CLU, and far too many others to list."
37. "V documentation (Introduction)" (<https://github.com/vlang/v/blob/master/doc/docs.md#introduction>). *GitHub*. Retrieved 24 December 2024.

38. Kuhlman, Dave. "A Python Book: Beginning Python, Advanced Python, and Python Exercises" (https://web.archive.org/web/20120623165941/http://cutter.rexx.com/~dkuhlman/python_book_01.html). Section 1.1. Archived from the original (https://www.davekuhlman.org/python_book_01.pdf) (PDF) on 23 June 2012.
39. "PEP 484 – Type Hints" (<https://peps.python.org/pep-0484/>). *Python Enhancement Proposals*. Retrieved 27 October 2025.
40. "mypy – Optional Static Typing for Python" (<https://mypy-lang.org/>). *mypy-lang.org*. Retrieved 17 August 2025.
41. "What's new in Python 3.15" (<https://docs.python.org/3.15/whatsnew/3.15.html>). Retrieved 26 January 2026.
42. Sultana, Simon G.; Reed, Philip A. (2017). "Curriculum for an Introductory Computer Science Course: Identifying Recommendations from Academia and Industry". *The Journal of Technology Studies*. **43** (2): 80–92. doi:10.21061/jots.v43i2.a.3 (<https://doi.org/10.21061%2Fjots.v43i2.a.3>). ISSN 1071-6084 (<https://search.worldcat.org/issn/1071-6084>). JSTOR 90023144 (<https://www.jstor.org/stable/90023144>).
43. van Rossum, Guido (20 January 2009). "A Brief Timeline of Python" (<https://python-history.blogspot.com/2009/01/brief-timeline-of-python.html>). *The History of Python*. Archived (<https://web.archive.org/web/20200605032200/https://python-history.blogspot.com/2009/01/brief-timeline-of-python.html>) from the original on 5 June 2020. Retrieved 20 January 2009.
44. van Rossum, Guido (29 August 2000). "SETL (was: Lukewarm about range literals)" (<https://mail.python.org/pipermail/python-dev/2000-August/008881.html>). *Python-Dev* (Mailing list). Archived (<https://web.archive.org/web/20180714064019/https://mail.python.org/pipermail/python-dev/2000-August/008881.html>) from the original on 14 July 2018. Retrieved 13 March 2011.
45. Fairchild, Carlie (12 July 2018). "Guido van Rossum Stepping Down from Role as Python's Benevolent Dictator For Life" (<https://www.linuxjournal.com/content/guido-van-rossum-stepping-down-role-pythons-benevolent-dictator-life>). *Linux Journal*. Archived (<https://web.archive.org/web/20180713192427/https://www.linuxjournal.com/content/guido-van-rossum-stepping-down-role-pythons-benevolent-dictator-life>) from the original on 13 July 2018. Retrieved 13 July 2018.
46. Smith, Nathaniel J.; Durbin, Ee. "PEP 8100 – January 2019 Steering Council election" (<https://peps.python.org/pep-8100/>). *Python Enhancement Proposals (PEPs)*. Python Software Foundation. Archived (<https://web.archive.org/web/20200604235027/https://www.python.org/dev/peps/pep-8100/>) from the original on 4 June 2020. Retrieved 4 May 2019.
47. The Python core team and community. "PEP 13 – Python Language Governance" (<https://peps.python.org/pep-0013/>). *Python Enhancement Proposals (PEPs)*. Archived (<https://web.archive.org/web/20210527000035/https://www.python.org/dev/peps/pep-0013/>) from the original on 27 May 2021. Retrieved 25 August 2021.
48. Briggs, Jason R.; Lipovača, Miran (2013). *Python for kids: a playful introduction to programming* (<https://archive.org/details/pythonforkidspla0000brig>). San Francisco, California, USA: No Starch Press. ISBN 978-1-59327-407-8. LCCN 2012044047 (<https://lccn.loc.gov/2012044047>). OCLC 825076499 (<https://search.worldcat.org/oclc/825076499>). OL 26119645M (<https://openlibrary.org/books/OL26119645M>).
49. Kuchling, A. M.; Zadka, Moshe (16 October 2000). "What's New in Python 2.0" (<https://docs.python.org/whatsnew/2.0.html>). Python Software Foundation. Archived (<https://web.archive.org/web/20121023112045/http://docs.python.org/whatsnew/2.0.html>) from the original on 23 October 2012. Retrieved 11 February 2012.
50. Peterson, Benjamin. "PEP 373 – Python 2.7 Release Schedule" (<https://legacy.python.org/dev/peps/pep-0373/>). *python.org*. Archived (<https://web.archive.org/web/20200519075520/https://legacy.python.org/dev/peps/pep-0373/>) from the original on 19 May 2020. Retrieved 9 January 2017.
51. Coghlan, Alyssa. "PEP 466 – Network Security Enhancements for Python 2.7.x" (<https://peps.python.org/pep-0466/>). *Python Enhancement Proposals (PEPs)*. Archived (<https://web.archive.org/web/20200604232833/https://www.python.org/dev/peps/pep-0466/>) from the original on 4 June 2020. Retrieved 9 January 2017.

52. "Sunsetting Python 2" (<https://www.python.org/doc/sunset-python-2/>). *Python.org*. Archived (<https://web.archive.org/web/20200112080903/https://www.python.org/doc/sunset-python-2/>) from the original on 12 January 2020. Retrieved 22 September 2019.
53. Peterson, Benjamin. "PEP 373 – Python 2.7 Release Schedule" (<https://peps.python.org/pep-0373/>). *Python Enhancement Proposals (PEPs)*. Archived (<https://web.archive.org/web/20200113033257/https://www.python.org/dev/peps/pep-0373/>) from the original on 13 January 2020. Retrieved 22 September 2019.
54. mattip (25 December 2023). "PyPy v7.3.14 release" (<https://www.pypy.org/posts/2023/12/pypy-v7314-release.html>). *PyPy*. Archived (<https://web.archive.org/web/20240105132820/https://www.pypy.org/posts/2023/12/pypy-v7314-release.html>) from the original on 5 January 2024. Retrieved 5 January 2024.
55. Peterson, Benjamin (20 April 2020). "Python 2.7.18, the last release of Python 2" (<https://pythoninsider.blogspot.com/2020/04/python-2718-last-release-of-python-2.html>). *Python Insider*. Archived (<https://web.archive.org/web/20200426204118/https://pythoninsider.blogspot.com/2020/04/python-2718-last-release-of-python-2.html>) from the original on 26 April 2020. Retrieved 27 April 2020.
56. "Status of Python versions" (<https://devguide.python.org/versions/>). *Python Developer's Guide*. Retrieved 12 November 2025.
57. The Cain Gang Ltd. "Python Metaclasses: Who? Why? When?" (<https://web.archive.org/web/20090530030205/http://www.python.org/community/pycon/dc2004/papers/24/metaclasses-pycon.pdf>) (PDF). Archived from the original (<https://www.python.org/community/pycon/dc2004/papers/24/metaclasses-pycon.pdf>) (PDF) on 30 May 2009. Retrieved 27 June 2009.
58. "3.3. Special method names" (<https://docs.python.org/3.0/reference/datamodel.html#special-method-names>). *The Python Language Reference*. Python Software Foundation. Archived (<https://web.archive.org/web/20181215123146/https://docs.python.org/3.0/reference/datamodel.html#special-method-names>) from the original on 15 December 2018. Retrieved 27 June 2009.
59. "PyDBC: method preconditions, method postconditions and class invariants for Python" (<http://www.nongnu.org/pydbc/>). Archived (<https://web.archive.org/web/20191123231931/http://www.nongnu.org/pydbc/>) from the original on 23 November 2019. Retrieved 24 September 2011.
60. "Contracts for Python" (<http://www.wayforward.net/pycontract/>). Archived (<https://web.archive.org/web/20200615173404/http://www.wayforward.net/pycontract/>) from the original on 15 June 2020. Retrieved 24 September 2011.
61. "PyDatalog" (<https://sites.google.com/site/pydatalog/>). Archived (<https://web.archive.org/web/20200613160231/https://sites.google.com/site/pydatalog/>) from the original on 13 June 2020. Retrieved 22 July 2012.
62. "Glue it all together with Python" (<https://www.python.org/doc/essays/omg-darpa-mcc-position/>). *Python.org*. Retrieved 30 September 2024.
63. "Reference counts" (<https://docs.python.org/extending/extending.html#reference-counts>). Extending and embedding the Python interpreter. *Docs.python.org*. Archived (<https://web.archive.org/web/20121018063230/http://docs.python.org/extending/extending.html#reference-counts>) from the original on 18 October 2012. Retrieved 5 June 2020. "Since Python makes heavy use of `malloc()` and `free()`, it needs a strategy to avoid memory leaks as well as the re-use of freed memory. The method chosen is called *reference counting*."
64. Hettinger, Raymond (30 January 2002). "PEP 289 – Generator Expressions" (<https://www.python.org/dev/peps/pep-0289/>). *Python Enhancement Proposals*. Python Software Foundation. Archived (<https://web.archive.org/web/20200614153717/https://www.python.org/dev/peps/pep-0289/>) from the original on 14 June 2020. Retrieved 19 February 2012.
65. "6.5 itertools – Functions creating iterators for efficient looping" (<https://docs.python.org/3/library/itertools.html>). *Docs.python.org*. Archived (<https://web.archive.org/web/20200614153629/https://docs.python.org/3/library/itertools.html>) from the original on 14 June 2020. Retrieved 22 November 2016.

66. Peters, Tim (19 August 2004). "PEP 20 – The Zen of Python" (<https://www.python.org/dev/peps/pep-0020/>). *Python Enhancement Proposals*. Python Software Foundation. Archived (<https://web.archive.org/web/20181226141127/https://www.python.org/dev/peps/pep-0020/>) from the original on 26 December 2018. Retrieved 24 November 2008.
67. Lutz, Mark (January 2022). "Python changes 2014+" (<https://learning-python.com/python-changes-2014-plus.html>). *Learning Python*. Archived (<https://web.archive.org/web/20240315075935/https://learning-python.com/python-changes-2014-plus.html>) from the original on 15 March 2024. Retrieved 25 February 2024.
68. "Confusion regarding a rule in 'the Zen of Python' " (<https://discuss.python.org/t/confusion-regarding-a-rule-in-the-zen-of-python/15927>). Discussions. *Python.org*. Python help. 3 May 2022. Archived (<https://web.archive.org/web/20240225221142/https://discuss.python.org/t/confusion-regarding-a-rule-in-the-zen-of-python/15927>) from the original on 25 February 2024. Retrieved 25 February 2024.
69. Ambi, Chetan (4 July 2021). "The most controversial Python 'walrus operator' " (<https://pythonsimplified.com/the-most-controversial-python-walrus-operator/>). *Python simplified* (*pythonsimplified.com*). Archived (<https://web.archive.org/web/20230827154931/https://pythonsimplified.com/the-most-controversial-python-walrus-operator/>) from the original on 27 August 2023. Retrieved 5 February 2024.
70. Grifski, Jeremy (24 May 2020). "The controversy behind the 'walrus operator' in Python" (<https://therenegadecoder.com/code/the-controversy-behind-the-walrus-operator-in-python/>). *The Renegade Coder* (*therenegadecoder.com*). Archived (<https://web.archive.org/web/20231228135749/https://therenegadecoder.com/code/the-controversy-behind-the-walrus-operator-in-python/>) from the original on 28 December 2023. Retrieved 25 February 2024.
71. van Kemenade, Hugo (2 June 2026). "Python 3.15.0 beta 2 is here!" (<https://blog.python.org/2026/06/python-3150-beta-2/>). *Python Insider*. Retrieved 18 June 2026.
72. Bader, Dan. "Python string formatting best practices" (<https://realpython.com/python-string-formatting/>). *Real Python* (*realpython.com*). Archived (<https://web.archive.org/web/20240218083506/https://realpython.com/python-string-formatting/>) from the original on 18 February 2024. Retrieved 25 February 2024.
73. Martelli, Alex; Ravenscroft, Anna; Ascher, David (2005). *Python Cookbook, 2nd Edition* (<http://shop.oreilly.com/product/9780596007973.do>). O'Reilly Media. p. 230. ISBN 978-0-596-00797-3. Archived (<https://web.archive.org/web/20200223171254/http://shop.oreilly.com/product/9780596007973.do>) from the original on 23 February 2020. Retrieved 14 November 2015.
74. "Python Culture" (<https://web.archive.org/web/20140130021902/http://ebeab.com/2014/01/21/python-culture/>). *ebeab*. 21 January 2014. Archived from the original (<http://ebeab.com/2014/01/21/python-culture/>) on 30 January 2014.
75. "Transpiling Python to Julia using PyJL" (https://web.ist.utl.pt/antonio.menezes.leitao/ADA/documents/publications_docs/2022_TranspilingPythonToJuliaUsingPyJL.pdf) (PDF). Archived (https://web.archive.org/web/20231119071525/https://web.ist.utl.pt/antonio.menezes.leitao/ADA/documents/publications_docs/2022_TranspilingPythonToJuliaUsingPyJL.pdf) (PDF) from the original on 19 November 2023. Retrieved 20 September 2023. "After manually modifying one line of code by specifying the necessary type information, we obtained a speedup of 52.6×, making the translated Julia code 19.5× faster than the original Python code."
76. "15 ways Python is a powerful force on the web" (<https://web.archive.org/web/20190511065650/http://insidetechnology.com/training/articles/8114-15-ways-python-is-a-powerful-force-on-the-web>). Archived from the original (<https://insidetechnology.com/training/articles/8114-15-ways-python-is-a-powerful-force-on-the-web>) on 11 May 2019. Retrieved 3 July 2018.
77. "Why is it called Python?" (<https://docs.python.org/3/faq/general.html#why-is-it-called-python>). *General Python FAQ*. Docs.python.org. Archived (<https://web.archive.org/web/20121024164224/http://docs.python.org/faq/general.html#why-is-it-called-python>) from the original on 24 October 2012. Retrieved 3 January 2023.
78. "pprint – data pretty printer – Python 3.11.0 documentation" (<https://docs.python.org/3/library/pprint.html>). *docs.python.org*. Archived (<https://web.archive.org/web/20210122224848/https://docs.python.org/3/library/pprint.html>) from the original on 22 January 2021. Retrieved 5 November 2022. "stuff = ['spam', 'eggs', 'lumberjack', 'knights', 'ni']"

79. "Code style" (<https://docs.python-guide.org/writing/style>). The hitchhiker's guide to Python. *docs.python-guide.org*. Archived (<https://web.archive.org/web/20210127154341/https://docs.python-guide.org/writing/style/>) from the original on 27 January 2021. Retrieved 20 January 2021.
80. Warsaw, Barry; Hylton, Jeremy; Goodder, David (13 June 2000). "PEP 1 – PEP Purpose and Guidelines" (<https://peps.python.org/pep-0001/>). *peps.python.org*. Retrieved 6 May 2026.
81. "Python Enhancement Proposal (PEP)" (<https://realpython.com/ref/glossary/pep/>). Retrieved 6 May 2026.
82. "What is a PEP in Python?" (<https://believemy.com/en/glossaries/python/pep>). *believemy.com*. Retrieved 7 May 2026.
83. "Is Python a good language for beginning programmers?" (<https://docs.python.org/faq/general.html#is-python-a-good-language-for-beginning-programmers>). *General Python FAQ*. Python Software Foundation. Archived (<https://web.archive.org/web/20121024164224/http://docs.python.org/faq/general.html#is-python-a-good-language-for-beginning-programmers>) from the original on 24 October 2012. Retrieved 21 March 2007.
84. "Myths about indentation in Python" (https://web.archive.org/web/20180218162410/http://www.secnex.de/~olli/Python/block_indentation.hawk). Secnetix.de. Archived from the original (http://www.secnex.de/~olli/Python/block_indentation.hawk) on 18 February 2018. Retrieved 19 April 2011.
85. Guttag, John V. (12 August 2016). *Introduction to Computation and Programming Using Python: With Application to Understanding Data*. MIT Press. ISBN 978-0-262-52962-4.
86. van Rossum, Guido; Warsaw, Barry. "PEP 8 – Style Guide for Python Code" (<https://www.python.org/dev/peps/pep-0008/>). *Python Enhancement Proposals (PEPs)*. Archived (<https://web.archive.org/web/20190417223549/https://www.python.org/dev/peps/pep-0008/>) from the original on 17 April 2019. Retrieved 26 March 2019.
87. "8. Errors and Exceptions – Python 3.12.0a0 documentation" (<https://docs.python.org/3.11/tutorial/errors.html>). *docs.python.org*. Archived (<https://web.archive.org/web/20220509145745/https://docs.python.org/3.11/tutorial/errors.html>) from the original on 9 May 2022. Retrieved 9 May 2022.
88. "Highlights: Python 2.5" (<https://www.python.org/download/releases/2.5/highlights/>). *Python.org*. Archived (<https://web.archive.org/web/20190804120408/https://www.python.org/download/releases/2.5/highlights/>) from the original on 4 August 2019. Retrieved 20 March 2018.
89. "What's new in Python 3.15" (<https://docs.python.org/3.15/whatsnew/3.15.html>). *Python documentation*. Retrieved 30 April 2026.
90. van Rossum, Guido (22 April 2009). "Tail Recursion Elimination" (<http://neopythonic.blogspot.be/2009/04/tail-recursion-elimination.html>). Neopythonic.blogspot.be. Archived (<https://web.archive.org/web/20180519225253/http://neopythonic.blogspot.be/2009/04/tail-recursion-elimination.html>) from the original on 19 May 2018. Retrieved 3 December 2012.
91. van Rossum, Guido (9 February 2006). "Language Design Is Not Just Solving Puzzles" (<http://www.artima.com/weblogs/viewpost.jsp?thread=147358>). *Artima forums*. Artima. Archived (<https://web.archive.org/web/20200117182525/https://www.artima.com/weblogs/viewpost.jsp?thread=147358>) from the original on 17 January 2020. Retrieved 21 March 2007.
92. van Rossum, Guido; Eby, Phillip J. (10 May 2005). "PEP 342 – Coroutines via Enhanced Generators" (<https://www.python.org/dev/peps/pep-0342/>). *Python Enhancement Proposals*. Python Software Foundation. Archived (<https://web.archive.org/web/20200529003739/https://www.python.org/dev/peps/pep-0342/>) from the original on 29 May 2020. Retrieved 19 February 2012.
93. "PEP 380" (<https://www.python.org/dev/peps/pep-0380/>). Python.org. Archived (<https://web.archive.org/web/20200604233821/https://www.python.org/dev/peps/pep-0380/>) from the original on 4 June 2020. Retrieved 3 December 2012.
94. "division" (<https://docs.python.org>). *python.org*. Archived (<https://web.archive.org/web/20060720033244/http://docs.python.org/>) from the original on 20 July 2006. Retrieved 30 July 2014.
95. "PEP 0465 – A dedicated infix operator for matrix multiplication" (<https://www.python.org/dev/peps/pep-0465/>). *python.org*. Archived (<https://web.archive.org/web/20200604224255/https://www.python.org/dev/peps/pep-0465/>) from the original on 4 June 2020. Retrieved 1 January 2016.

96. "Python 3.5.1 Release and Changelog" (<https://www.python.org/downloads/release/python-351/>). *python.org*. Archived (<https://web.archive.org/web/20200514034938/https://www.python.org/downloads/release/python-351/>) from the original on 14 May 2020. Retrieved 1 January 2016.
97. "What's New in Python 3.8" (<https://docs.python.org/3.8/whatsnew/3.8.html>). Archived (<https://web.archive.org/web/20200608124345/https://docs.python.org/3.8/whatsnew/3.8.html>) from the original on 8 June 2020. Retrieved 14 October 2019.
98. van Rossum, Guido; Hettinger, Raymond (7 February 2003). "PEP 308 – Conditional Expressions" (<https://www.python.org/dev/peps/pep-0308/>). *Python Enhancement Proposals*. Python Software Foundation. Archived (<https://web.archive.org/web/20160313113147/https://www.python.org/dev/peps/pep-0308/>) from the original on 13 March 2016. Retrieved 13 July 2011.
99. "4. Built-in Types – Python 3.6.3rc1 documentation" (<https://docs.python.org/3/library/stdtypes.html#tuple>). *python.org*. Archived (<https://web.archive.org/web/20200614194325/https://docs.python.org/3/library/stdtypes.html#tuple>) from the original on 14 June 2020. Retrieved 1 October 2017.
100. "5.3. Tuples and Sequences – Python 3.7.1rc2 documentation" (<https://docs.python.org/3/tutorial/datastructures.html#tuples-and-sequences>). *python.org*. Archived (<https://web.archive.org/web/20200610050047/https://docs.python.org/3/tutorial/datastructures.html#tuples-and-sequences>) from the original on 10 June 2020. Retrieved 17 October 2018.
101. "PEP 498 – Literal String Interpolation" (<https://www.python.org/dev/peps/pep-0498/>). *python.org*. Archived (<https://web.archive.org/web/20200615184141/https://www.python.org/dev/peps/pep-0498/>) from the original on 15 June 2020. Retrieved 8 March 2017.
102. "The Python Language Reference, section 3.3. New-style and classic classes, for release 2.7.1" (<https://web.archive.org/web/20121026063834/http://docs.python.org/reference/datamodel.html#new-style-and-classic-classes>). Archived from the original (<https://docs.python.org/reference/datamodel.html#new-style-and-classic-classes>) on 26 October 2012. Retrieved 12 January 2011.
103. "PEP 484 – Type Hints | peps.python.org" (<https://peps.python.org/pep-0484/>). *peps.python.org*. Archived (<https://web.archive.org/web/20231127205023/https://peps.python.org/pep-0484/>) from the original on 27 November 2023. Retrieved 29 November 2023.
104. "typing — Support for type hints" (<https://docs.python.org/3/library/typing.html>). *Python documentation*. Python Software Foundation. Archived (<https://web.archive.org/web/20200221184042/https://docs.python.org/3/library/typing.html>) from the original on 21 February 2020. Retrieved 22 December 2023.
105. "mypy – Optional Static Typing for Python" (<http://mypy-lang.org/>). Archived (<https://web.archive.org/web/20200606192012/http://mypy-lang.org/>) from the original on 6 June 2020. Retrieved 28 January 2017.
106. "Introduction" (<https://mypy.readthedocs.io/en/latest/introduction.html>). *mypy.readthedocs.io*. Archived (<https://web.archive.org/web/20231222000457/https://mypy.readthedocs.io/en/latest/introduction.html>) from the original on 22 December 2023. Retrieved 22 December 2023.
107. "on what systems does Python not use IEEE-754 double precision floats" (<https://stackoverflow.com/questions/70184494/on-what-systems-does-python-not-use-ieee-754-double-precision-floats/70189136#70189136>). *Stack Overflow*. Retrieved 7 June 2026.
108. "15. Floating Point Arithmetic: Issues and Limitations – Python 3.8.3 documentation" (<https://docs.python.org/3.8/tutorial/floatpoint.html#representation-error>). *docs.python.org*. Archived (<https://web.archive.org/web/20200606113842/https://docs.python.org/3.8/tutorial/floatpoint.html#representation-error>) from the original on 6 June 2020. Retrieved 6 June 2020. "Almost all machines today (November 2000) use IEEE-754 floating point arithmetic, and almost all platforms map Python floats to IEEE-754 "double precision"."
109. "ArrowNotImplementedError: halffloat error on applying pandas.to_feather on a dataframe" (<https://stackoverflow.com/questions/54150865/arrownotimplementederror-halffloat-error-on-applying-pandas-to-feather-on-a-dat>). *Stack Overflow*. Retrieved 8 June 2026.

110. "Exotic Floating Point Formats — bitstring 4.3 documentation" (https://bitstring.readthedocs.io/en/stable/exotic_floats.html). *bitstring.readthedocs.io*. Retrieved 8 June 2026.
111. Zadka, Moshe; van Rossum, Guido (11 March 2001). "PEP 237 – Unifying Long Integers and Integers" (<https://www.python.org/dev/peps/pep-0237/>). *Python Enhancement Proposals*. Python Software Foundation. Archived (<https://web.archive.org/web/20200528063237/https://www.python.org/dev/peps/pep-0237/>) from the original on 28 May 2020. Retrieved 24 September 2011.
112. "Built-in Types" (<https://docs.python.org/3/library/stdtypes.html#typeseq-range>). Archived (<https://web.archive.org/web/20200614194325/https://docs.python.org/3/library/stdtypes.html#typeseq-range>) from the original on 14 June 2020. Retrieved 3 October 2019.
113. "PEP 465 – A dedicated infix operator for matrix multiplication" (<https://legacy.python.org/dev/peps/pep-0465/>). *python.org*. Archived (<https://web.archive.org/web/20200529200310/https://legacy.python.org/dev/peps/pep-0465/>) from the original on 29 May 2020. Retrieved 3 July 2018.
114. Zadka, Moshe; van Rossum, Guido (11 March 2001). "PEP 238 – Changing the Division Operator" (<https://www.python.org/dev/peps/pep-0238/>). *Python Enhancement Proposals*. Python Software Foundation. Archived (<https://web.archive.org/web/20200528115550/https://www.python.org/dev/peps/pep-0238/>) from the original on 28 May 2020. Retrieved 23 October 2013.
115. "Why Python's Integer Division Floors" (<https://python-history.blogspot.com/2010/08/why-pythons-integer-division-floors.html>). 24 August 2010. Archived (<https://web.archive.org/web/20200605151500/https://python-history.blogspot.com/2010/08/why-pythons-integer-division-floors.html>) from the original on 5 June 2020. Retrieved 25 August 2010.
116. "round" (<https://docs.python.org/py3k/library/functions.html#round>), *The Python standard library, release 3.2, §2: Built-in functions*, archived (<https://web.archive.org/web/20121025141808/http://docs.python.org/py3k/library/functions.html#round>) from the original on 25 October 2012, retrieved 14 August 2011
117. "round" (<https://docs.python.org/library/functions.html#round>), *The Python standard library, release 2.7, §2: Built-in functions*, archived (<https://web.archive.org/web/20121027081602/http://docs.python.org/library/functions.html#round>) from the original on 27 October 2012, retrieved 14 August 2011
118. Beazley, David M. (2009). *Python Essential Reference* (https://archive.org/details/pythonessentialr00beaz_036) (4th ed.). Addison-Wesley Professional. p. 66 (https://archive.org/details/pythonessentialr00beaz_036/page/n90). ISBN 978-0-672-32978-4.
119. Kernighan, Brian W.; Ritchie, Dennis M. (1988). *The C Programming Language* (2nd ed.). p. 206 (<https://archive.org/details/cprogramminglang00bria/page/206>).
120. Batista, Facundo (17 October 2003). "PEP 327 – Decimal Data Type" (<https://www.python.org/dev/peps/pep-0327/>). *Python Enhancement Proposals*. Python Software Foundation. Archived (<https://web.archive.org/web/20200604234830/https://www.python.org/dev/peps/pep-0327/>) from the original on 4 June 2020. Retrieved 24 November 2008.
121. "What's New in Python 2.6" (<https://docs.python.org/2.6/whatsnew/2.6.html>). *Python v2.6.9 documentation*. 29 October 2013. Archived (<https://web.archive.org/web/20191223213856/https://docs.python.org/2.6/whatsnew/2.6.html>) from the original on 23 December 2019. Retrieved 26 September 2015.
122. "10 Reasons Python Rocks for Research (And a Few Reasons it Doesn't) – Hoyt Koepke" (<https://web.archive.org/web/20200531211840/https://www.stat.washington.edu/~hoytak/blog/whypython.html>). *University of Washington Department of Statistics*. Archived from the original (<https://www.stat.washington.edu/~hoytak/blog/whypython.html>) on 31 May 2020. Retrieved 3 February 2019.
123. Shell, Scott (17 June 2014). "An introduction to Python for scientific computing" (<https://engineering.ucsb.edu/~shell/che210d/python.pdf>) (PDF). Archived (<https://web.archive.org/web/20190204014642/https://engineering.ucsb.edu/~shell/che210d/python.pdf>) (PDF) from the original on 4 February 2019. Retrieved 3 February 2019.

124. Piotrowski, Przemyslaw (July 2006). "Build a Rapid Web Development Environment for Python Server Pages and Oracle" (<http://www.oracle.com/technetwork/articles/piotrowski-pythoncore-084049.html>). *Oracle Technology Network*. Oracle. Archived (<https://web.archive.org/web/20190402124435/https://www.oracle.com/technetwork/articles/piotrowski-pythoncore-084049.html>) from the original on 2 April 2019. Retrieved 12 March 2012.
125. Eby, Phillip J. (7 December 2003). "PEP 333 – Python Web Server Gateway Interface v1.0" (<https://www.python.org/dev/peps/pep-0333/>). *Python Enhancement Proposals*. Python Software Foundation. Archived (<https://web.archive.org/web/20200614170344/https://www.python.org/dev/peps/pep-0333/>) from the original on 14 June 2020. Retrieved 19 February 2012.
126. "PyPI" (<https://pypi.org/>). *PyPI*. 13 March 2025. Archived (<https://web.archive.org/web/20250222013445/https://pypi.org/>) from the original on 22 February 2025.
127. "Glossary: interactive" (<https://docs.python.org/3/glossary.html#term-interactive>). *Python documentation*. v3.13.7. Retrieved 31 August 2025.
128. "IDLE — Python editor and shell" (<https://docs.python.org/3/library/idle.html>). *Python documentation*. v3.13.7. Retrieved 31 August 2025. "IDLE is Python's Integrated Development and Learning Environment."
129. "IPython Documentation" (<https://ipython.readthedocs.io/en/stable/>). v9.5.0. 29 August 2025. Archived (<https://web.archive.org/web/20250831204721/https://ipython.readthedocs.io/en/stable/>) from the original on 31 August 2025. Retrieved 31 August 2025.
130. "Python in Visual Studio Code" (<https://code.visualstudio.com/docs/languages/python>). *code.visualstudio.com*. Retrieved 1 December 2025.
131. "Project Jupyter" (<https://jupyter.org>). *Jupyter.org*. Archived (<https://web.archive.org/web/20231012055917/https://jupyter.org/>) from the original on 12 October 2023. Retrieved 2 April 2025.
132. Harper, Doug (Spring 2024). "Enthought Canopy" (<http://physics.wku.edu/phys316/software/canopy/>). *WKU Physics 316*. Western Kentucky University. Archived (<https://web.archive.org/web/20240818041226/http://physics.wku.edu/phys316/software/canopy/>) from the original on 18 August 2024. Retrieved 31 August 2025.
133. "Enthought Canopy" (<https://web.archive.org/web/20170715151703/https://www.enthought.com/products/canopy/>). *Enthought*. Archived from the original (<https://www.enthought.com/products/canopy/>) on 15 July 2017. Retrieved 20 August 2016.
134. "PEP 7 – Style Guide for C Code | peps.python.org" (<https://peps.python.org/pep-0007/>). *peps.python.org*. Archived (<https://web.archive.org/web/20220424202827/https://peps.python.org/pep-0007/>) from the original on 24 April 2022. Retrieved 28 April 2022.
135. "4. Building C and C++ Extensions – Python 3.9.2 documentation" (<https://docs.python.org/3/extending/building.html>). *docs.python.org*. Archived (<https://web.archive.org/web/20210303002519/https://docs.python.org/3/extending/building.html>) from the original on 3 March 2021. Retrieved 1 March 2021.
136. van Rossum, Guido (5 June 2001). "PEP 7 – Style Guide for C Code" (<https://www.python.org/dev/peps/pep-0007/>). *Python Enhancement Proposals*. Python Software Foundation. Archived (<https://web.archive.org/web/20200601203908/https://www.python.org/dev/peps/pep-0007/>) from the original on 1 June 2020. Retrieved 24 November 2008.
137. "CPython byte code" (<https://docs.python.org/3/library/dis.html#python-bytecode-instructions>). *Docs.python.org*. Archived (<https://web.archive.org/web/20200605151542/https://docs.python.org/3/library/dis.html#python-bytecode-instructions>) from the original on 5 June 2020. Retrieved 16 February 2016.
138. "Python 2.5 internals" (<http://www.troeger.eu/teaching/pythonvm08.pdf>) (PDF). Archived (<https://web.archive.org/web/20120806094951/http://www.troeger.eu/teaching/pythonvm08.pdf>) (PDF) from the original on 6 August 2012. Retrieved 19 April 2011.
139. "Changelog – Python 3.9.0 documentation" (<https://docs.python.org/release/3.9.0/whatsnew/changelog.html#changelog>). *docs.python.org*. Archived (<https://web.archive.org/web/20210207001142/https://docs.python.org/release/3.9.0/whatsnew/changelog.html#changelog>) from the original on 7 February 2021. Retrieved 8 February 2021.

140. "Download Python" (<https://www.python.org/downloads/release/python-391/>). *Python.org*. Archived (<https://web.archive.org/web/20201208045225/https://www.python.org/downloads/release/python-391/>) from the original on 8 December 2020. Retrieved 13 December 2020.
141. "VMS Python compatibility issues - VSI OpenVMS Wiki" (https://wiki.vmssoftware.com/VMS_Python_compatibility_issues). *wiki.vmssoftware.com*. Retrieved 8 June 2026.
142. "Python and Python Wheels for OpenVMS — VMS Software, Inc" (<https://docs.vmssoftware.com/python-v3-10-10-and-wheels-v1-1-7-for-openvms-release-notes/>). *docs.vmssoftware.com*. Retrieved 8 June 2026.
143. "history [vmspython]" (<https://www.vmspython.org/doku.php?id=history>). *www.vmspython.org*. Archived (<https://web.archive.org/web/20201202194743/https://www.vmspython.org/doku.php?id=history>) from the original on 2 December 2020. Retrieved 4 December 2020.
144. "An Interview with Guido van Rossum" (http://www.oreilly.com/pub/a/oreilly/frank/rosum_1099.html). *Oreilly.com*. Archived (https://web.archive.org/web/20140716222652/http://oreilly.com/pub/a/oreilly/frank/rosum_1099.html) from the original on 16 July 2014. Retrieved 24 November 2008.
145. Pereira, Rui; Couto, Marco; Ribeiro, Francisco; Rua, Rui; Cunha, Jácome; Fernandes, João Paulo; Saraiva, João (23 October 2017). "Energy efficiency across programming languages: How do energy, time, and memory relate?" (<https://doi.org/10.1145/3136014.3136031>). *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering* (<http://repositorio.inesctec.pt/handle/123456789/5492>). SLE 2017. New York, NY, USA: Association for Computing Machinery. pp. 256–267. doi:10.1145/3136014.3136031 (<https://doi.org/10.1145/3136014.3136031>). ISBN 978-1-4503-5525-4.
146. "What PyInstaller Does and How It Does It" (<https://pyinstaller.org/en/stable/operating-mode.html>).
147. "PyPy compatibility" (<https://pypy.org/compat.html>). *Pypy.org*. Archived (<https://web.archive.org/web/20200606041845/https://www.pypy.org/compat.html>) from the original on 6 June 2020. Retrieved 3 December 2012.
148. Team, The PyPy (28 December 2019). "Download and Install" (<https://www.pypy.org/download.html>). *PyPy*. Archived (<https://web.archive.org/web/20220108212951/https://www.pypy.org/download.html>) from the original on 8 January 2022. Retrieved 8 January 2022.
149. "speed comparison between CPython and Pypy" (<https://speed.pypy.org/>). *Speed.pypy.org*. Archived (<https://web.archive.org/web/20210510014902/https://speed.pypy.org/>) from the original on 10 May 2021. Retrieved 3 December 2012.
150. "Codon: Differences with Python" (<https://docs.exaloop.io/codon/general/differences>). Archived (<https://web.archive.org/web/20230525002540/https://docs.exaloop.io/codon/general/differences>) from the original on 25 May 2023. Retrieved 28 August 2023.
151. Lawson, Loraine (14 March 2023). "MIT-Created Compiler Speeds up Python Code" (<https://thenewstack.io/mit-created-compiler-speeds-up-python-code/>). *The New Stack*. Archived (<https://web.archive.org/web/20230406054200/https://thenewstack.io/mit-created-compiler-speeds-up-python-code/>) from the original on 6 April 2023. Retrieved 28 August 2023.
152. "Python-for-EV3" (<https://education.lego.com/en-us/support/mindstorms-ev3/python-for-ev3>). *LEGO Education*. Archived (<https://web.archive.org/web/20200607234814/https://education.lego.com/en-us/support/mindstorms-ev3/python-for-ev3>) from the original on 7 June 2020. Retrieved 17 April 2019.
153. Yegulalp, Serdar (29 October 2020). "Pyston returns from the dead to speed Python" (<https://www.infoworld.com/article/3587591/pyston-returns-from-the-dead-to-speed-python.html>). *InfoWorld*. Archived (<https://web.archive.org/web/20210127113233/https://www.infoworld.com/article/3587591/pyston-returns-from-the-dead-to-speed-python.html>) from the original on 27 January 2021. Retrieved 26 January 2021.
154. "cinder: Instagram's performance-oriented fork of CPython" (<https://github.com/facebookincubator/cinder>). *GitHub*. Archived (<https://web.archive.org/web/20210504112500/https://github.com/facebookincubator/cinder>) from the original on 4 May 2021. Retrieved 4 May 2021.

155. Aroca, Rafael (7 August 2021). "Snek Lang: feels like Python on Arduinos" (<https://rafaelaroca.wordpress.com/2021/08/07/snek-lang-feels-like-python-on-arduinosaurs/>). *Yet Another Technology Blog*. Archived (<https://web.archive.org/web/20240105001031/https://rafaelaroca.wordpress.com/2021/08/07/snek-lang-feels-like-python-on-arduinosaurs/>) from the original on 5 January 2024. Retrieved 4 January 2024.
156. Aufranc (CNXSoft), Jean-Luc (16 January 2020). "Snekboard Controls LEGO Power Functions with CircuitPython or Snek Programming Languages (Crowdfunding) – CNX Software" (<https://www.cnx-software.com/2020/01/16/snekboard-controls-lego-power-functions-with-circuitpython-or-snek-programming-languages/>). *CNX Software – Embedded Systems News*. Archived (<https://web.archive.org/web/20240105001031/https://www.cnx-software.com/2020/01/16/snekboard-controls-lego-power-functions-with-circuitpython-or-snek-programming-languages/>) from the original on 5 January 2024. Retrieved 4 January 2024.
157. Kennedy (@mkennedy), Michael. "Ready to find out if you're git famous?" (<https://pythonbytes.fm/episodes/show/187/ready-to-find-out-if-youre-git-famous>). *pythonbytes.fm*. Archived (<https://web.archive.org/web/20240105001031/https://pythonbytes.fm/episodes/show/187/ready-to-find-out-if-youre-git-famous>) from the original on 5 January 2024. Retrieved 4 January 2024.
158. Packard, Keith (20 December 2022). "The Snek Programming Language: A Python-inspired Embedded Computing Language" (<https://sneklang.org/doc/snek.pdf>) (PDF). Archived (<https://web.archive.org/web/20240104162458/https://sneklang.org/doc/snek.pdf>) (PDF) from the original on 4 January 2024. Retrieved 4 January 2024.
159. *RustPython/RustPython* (<https://github.com/RustPython/RustPython>), RustPython Dev, 5 May 2026, retrieved 6 May 2026
160. "Application-level Stackless features – PyPy 2.0.2 documentation" (<https://doc.pypy.org/en/latest/stackless.html>). Doc.pypy.org. Archived (<https://web.archive.org/web/20200604231513/https://doc.pypy.org/en/latest/stackless.html>) from the original on 4 June 2020. Retrieved 17 July 2013.
161. "Plans for optimizing Python" (<https://code.google.com/p/unladen-swallow/wiki/ProjectPlan>). *Google Project Hosting*. 15 December 2009. Archived (<https://web.archive.org/web/2016041181848/https://code.google.com/p/unladen-swallow/wiki/ProjectPlan>) from the original on 11 April 2016. Retrieved 24 September 2011.
162. "Python on the Nokia N900" (<http://www.stochasticgeometry.ie/2010/04/29/python-on-the-nokia-n900/>). *Stochastic Geometry*. 29 April 2010. Archived (<https://web.archive.org/web/2019062000053/http://www.stochasticgeometry.ie/2010/04/29/python-on-the-nokia-n900/>) from the original on 20 June 2019. Retrieved 9 July 2015.
163. "Brython" (<https://brython.info/>). *brython.info*. Archived (<https://web.archive.org/web/20180803065954/http://brython.info/>) from the original on 3 August 2018. Retrieved 21 January 2021.
164. "Transcrypt – Python in the browser" (<https://www.transcrypt.org>). *transcrypt.org*. Archived (<https://web.archive.org/web/20180819133303/http://www.transcrypt.org/>) from the original on 19 August 2018. Retrieved 22 December 2020.
165. "Transcrypt: Anatomy of a Python to JavaScript Compiler" (<https://www.infoq.com/articles/transcrypt-python-javascript-compiler/>). *InfoQ*. Archived (<https://web.archive.org/web/20201205193339/https://www.infoq.com/articles/transcrypt-python-javascript-compiler/>) from the original on 5 December 2020. Retrieved 20 January 2021.
166. "Nuitka Home | Nuitka Home" (<https://nuitka.net/>). *nuitka.net*. Archived (<https://web.archive.org/web/20200530211233/https://nuitka.net/>) from the original on 30 May 2020. Retrieved 18 August 2017.
167. Guelton, Serge; Brunet, Pierrick; Amini, Mehdi; Merlini, Adrien; Corbillon, Xavier; Raynaud, Alan (16 March 2015). "Pythran: enabling static optimization of scientific Python programs" (<https://doi.org/10.1088%2F1749-4680%2F8%2F1%2F014001>). *Computational Science & Discovery*. **8** (1) 014001. IOP Publishing. Bibcode:2015CS&D....8a4001G (<https://ui.adsabs.harvard.edu/abs/2015CS&D....8a4001G>). doi:10.1088/1749-4680/8/1/014001 (<https://doi.org/10.1088%2F1749-4680%2F8%2F1%2F014001>). ISSN 1749-4699 (<https://search.worldcat.org/issn/1749-4699>).

168. "The Python → 11l → C++ transpiler" (<https://11l-lang.org/transpiler/>). Archived (<https://web.archive.org/web/20220924233728/https://11l-lang.org/transpiler/>) from the original on 24 September 2022. Retrieved 17 July 2022.
169. "google/grumpy" (<https://github.com/google/grumpy>). 10 April 2020. Archived (<https://web.archive.org/web/20200415054919/https://github.com/google/grumpy>) from the original on 15 April 2020. Retrieved 25 March 2020 – via GitHub.
170. "Projects" (<https://opensource.google/projects/>). *opensource.google*. Archived (<https://web.archive.org/web/20200424191248/https://opensource.google/projects/>) from the original on 24 April 2020. Retrieved 25 March 2020.
171. Francisco, Thomas Claburn in San. "Google's Grumpy code makes Python Go" (https://www.theregister.com/2017/01/05/googles_grumpy_makes_python_go/). *www.theregister.com*. Archived (https://web.archive.org/web/20210307165521/https://www.theregister.com/2017/01/05/googles_grumpy_makes_python_go/) from the original on 7 March 2021. Retrieved 20 January 2021.
172. "IronPython.net /" (<https://ironpython.net/>). *ironpython.net*. Archived (<https://web.archive.org/web/20210417064418/https://ironpython.net/>) from the original on 17 April 2021.
173. "GitHub – IronLanguages/ironpython3: Implementation of Python 3.x for .NET Framework that is built on top of the Dynamic Language Runtime" (<https://github.com/IronLanguages/ironpython3>). *GitHub*. Archived (<https://web.archive.org/web/20210928101250/https://github.com/IronLanguages/ironpython3>) from the original on 28 September 2021.
174. "Jython FAQ" (<https://www.jython.org/jython-old-sites/archive/22/userfaq.html>). *www.jython.org*. Archived (<https://web.archive.org/web/20210422055726/https://www.jython.org/jython-old-sites/archive/22/userfaq.html>) from the original on 22 April 2021. Retrieved 22 April 2021.
175. Murri, Riccardo (2013). *Performance of Python runtimes on a non-numeric scientific code*. European Conference on Python in Science (EuroSciPy). arXiv:1404.6388 (<https://arxiv.org/abs/1404.6388>). Bibcode:2014arXiv1404.6388M (<https://ui.adsabs.harvard.edu/abs/2014arXiv1404.6388M>).
176. "The Computer Language Benchmarks Game" (<https://benchmarksgame-team.pages.debian.net/benchmarksgame/fastest/python.html>). Archived (<https://web.archive.org/web/20200614210246/https://benchmarksgame-team.pages.debian.net/benchmarksgame/fastest/python.html>) from the original on 14 June 2020. Retrieved 30 April 2020.
177. Python, Real. "Look Ma, No for Loops: Array Programming With NumPy – Real Python" (<https://realpython.com/numpy-array-programming/>). *realpython.com*. Retrieved 15 October 2025.
178. Warsaw, Barry; Hylton, Jeremy; Goodger, David (13 June 2000). "PEP 1 – PEP Purpose and Guidelines" (<https://www.python.org/dev/peps/pep-0001/>). *Python Enhancement Proposals*. Python Software Foundation. Archived (<https://web.archive.org/web/20200606042011/https://www.python.org/dev/peps/pep-0001/>) from the original on 6 June 2020. Retrieved 19 April 2011.
179. Cannon, Brett. "Guido, Some Guys, and a Mailing List: How Python is Developed" (<https://web.archive.org/web/20090601134342/http://www.python.org/dev/intro/>). *python.org*. Python Software Foundation. Archived from the original (<https://www.python.org/dev/intro/>) on 1 June 2009. Retrieved 27 June 2009.
180. Edge, Jake (23 February 2022). "Moving Python's bugs to GitHub [LWN.net]" (<https://lwn.net/Articles/885854/>). Archived (<https://web.archive.org/web/20221002183818/https://lwn.net/Articles/885854/>) from the original on 2 October 2022. Retrieved 2 October 2022.
181. "Python Developer's Guide – Python Developer's Guide" (<https://devguide.python.org/>). *devguide.python.org*. Archived (<https://web.archive.org/web/20201109032501/https://devguide.python.org/>) from the original on 9 November 2020. Retrieved 17 December 2019.
182. Hughes, Owen (24 May 2021). "Programming languages: Why Python 4.0 might never arrive, according to its creator" (<https://www.techrepublic.com/article/programming-languages-why-python-4-0-will-probably-never-arrive-according-to-its-creator/>). *TechRepublic*. Archived (<https://web.archive.org/web/20220714201302/https://www.techrepublic.com/article/programming-languages-why-python-4-0-will-probably-never-arrive-according-to-its-creator/>) from the original on 14 July 2022. Retrieved 16 May 2022.

183. "PEP 602 – Annual Release Cycle for Python" (<https://www.python.org/dev/peps/pep-0602/>). *Python.org*. Archived (<https://web.archive.org/web/20200614202755/https://www.python.org/dev/peps/pep-0602/>) from the original on 14 June 2020. Retrieved 6 November 2019.
184. Edge, Jake (23 October 2019). "Changing the Python release cadence [LWN.net]" (<https://lwn.net/Articles/802777/>). *lwn.net*. Archived (<https://web.archive.org/web/20191106170153/https://lwn.net/Articles/802777/>) from the original on 6 November 2019. Retrieved 6 November 2019.
185. Norwitz, Neal (8 April 2002). "[Python-Dev] Release Schedules (was Stability & change)" (<https://mail.python.org/pipermail/python-dev/2002-April/022739.html>). Archived (<https://web.archive.org/web/20181215122750/https://mail.python.org/pipermail/python-dev/2002-April/022739.html>) from the original on 15 December 2018. Retrieved 27 June 2009.
186. Aahz; Baxter, Anthony (15 March 2001). "PEP 6 – Bug Fix Releases" (<https://www.python.org/dev/peps/pep-0006/>). *Python Enhancement Proposals*. Python Software Foundation. Archived (<https://web.archive.org/web/20200605001318/https://www.python.org/dev/peps/pep-0006/>) from the original on 5 June 2020. Retrieved 27 June 2009.
187. "Python Buildbot" (<https://www.python.org/dev/buildbot/>). *Python Developer's Guide*. Python Software Foundation. Archived (<https://web.archive.org/web/20200605001322/https://www.python.org/dev/buildbot/>) from the original on 5 June 2020. Retrieved 24 September 2011.
188. "Whetting Your Appetite" (<https://docs.python.org/tutorial/appetite.html>). *The Python Tutorial*. Python Software Foundation. Archived (<https://web.archive.org/web/20121026063559/http://docs.python.org/tutorial/appetite.html>) from the original on 26 October 2012. Retrieved 20 February 2012.
189. "In Python, should I use else after a return in an if block?" (<https://stackoverflow.com/questions/5033906/in-python-should-i-use-else-after-a-return-in-an-if-block>). *Stack Overflow*. Stack Exchange. 17 February 2011. Archived (<https://web.archive.org/web/20190620000050/https://stackoverflow.com/questions/5033906/in-python-should-i-use-else-after-a-return-in-an-if-block>) from the original on 20 June 2019. Retrieved 6 May 2011.
190. Lutz 2013, p. 17.
191. Fehily, Chris (2002). *Python* (<https://books.google.com/books?id=carqldfVIYC&pg=PR15>). Peachpit Press. p. xv. ISBN 978-0-201-74884-0. Archived (<https://web.archive.org/web/20170717044040/https://books.google.com/books?id=carqldfVIYC&pg=PR15>) from the original on 17 July 2017. Retrieved 9 May 2017.
192. Lubanovic, Bill (2014). *Introducing Python* (https://archive.org/details/introducingpytho0000lub_a). Sebastopol, CA : O'Reilly Media. p. 305. ISBN 978-1-4493-5936-2. Retrieved 31 July 2023.
193. Esterbrook, Charles. "Acknowledgements" (<https://web.archive.org/web/20080208141002/http://cobra-language.com/docs/acknowledgements/>). *cobra-language.com*. Cobra Language. Archived from the original (<http://cobra-language.com/docs/acknowledgements/>) on 8 February 2008. Retrieved 7 April 2010.
194. "Proposals: iterators and generators [ES4 Wiki]" (https://web.archive.org/web/20071020082650/http://wiki.ecmascript.org/doku.php?id=proposals:iterators_and_generators). *wiki.ecmascript.org*. Archived from the original (http://wiki.ecmascript.org/doku.php?id=proposals:iterators_and_generators) on 20 October 2007. Retrieved 24 November 2008.
195. Kincaid, Jason (10 November 2009). "Google's Go: A New Programming Language That's Python Meets C++" (<https://techcrunch.com/2009/11/10/google-go-language/>). *TechCrunch*. Archived (<https://web.archive.org/web/20100118014358/http://www.techcrunch.com/2009/11/10/google-go-language/>) from the original on 18 January 2010. Retrieved 29 January 2010.
196. "Why We Created Julia" (<https://julialang.org/blog/2012/02/why-we-created-julia>). *Julia website*. February 2012. Archived (<https://web.archive.org/web/20200502144010/https://julialang.org/blog/2012/02/why-we-created-julia/>) from the original on 2 May 2020. Retrieved 5 June 2014. "We want something as usable for general programming as Python [...]"

197. "Modular Docs – Why Mojo" (<https://docs.modular.com/mojo/why-mojo.html>). *docs.modular.com*. Archived (<https://web.archive.org/web/20230505083518/https://docs.modular.com/mojo/why-mojo.html>) from the original on 5 May 2023. Retrieved 5 May 2023. "Mojo as a member of the Python family [...] Embracing Python massively simplifies our design efforts, because most of the syntax is already specified. [...] we decided that the right long-term goal for Mojo is to provide a superset of Python (i.e. be compatible with existing programs) and to embrace the CPython immediately for long-tail ecosystem enablement. To a Python programmer, we expect and hope that Mojo will be immediately familiar, while also providing new tools for developing systems-level code that enable you to do things that Python falls back to C and C++ for."
198. Spencer, Michael (4 May 2023). "What is Mojo Programming Language?" (<https://datasciencelearningcenter.substack.com/p/what-is-mojo-programming-language>). *datasciencelearningcenter.substack.com*. Archived (<https://web.archive.org/web/20230505090408/https://datasciencelearningcenter.substack.com/p/what-is-mojo-programming-language>) from the original on 5 May 2023. Retrieved 5 May 2023.
199. "GDScript" (<https://gdscript.com/>). *gdscript.com*. Retrieved 24 November 2025.

Sources

- "Python for Artificial Intelligence" (<https://web.archive.org/web/20121101045354/http://wiki.python.org/moin/PythonForArtificialIntelligence>). Python Wiki. 19 July 2012. Archived from the original (<https://wiki.python.org/moin/PythonForArtificialIntelligence>) on 1 November 2012. Retrieved 3 December 2012.
- Paine, Jocelyn, ed. (August 2005). "AI in Python" (https://web.archive.org/web/20120326105810/http://www.ainewsletter.com/newsletters/aix_0508.htm#python_ai_ai). *AI Expert Newsletter*. Amzi!. Archived from the original (http://www.ainewsletter.com/newsletters/aix_0508.htm#python_ai_ai) on 26 March 2012. Retrieved 11 February 2012.
- "PyAIML 0.8.5: Python Package Index" (<https://pypi.python.org/pypi/PyAIML>). Pypi.python.org. Retrieved 17 July 2013.
- Russell, Stuart J. & Norvig, Peter (2009). *Artificial Intelligence: A Modern Approach* (3rd ed.). Upper Saddle River, NJ: Prentice Hall. ISBN 978-0-13-604259-4.

Further reading

- Downey, Allen (July 2024). *Think Python: How to Think Like a Computer Scientist* (<https://alendowney.github.io/ThinkPython/>) (3rd ed.). O'Reilly Media. ISBN 978-1-0981-5543-8.
- Lutz, Mark (2013). *Learning Python* (5th ed.). O'Reilly Media. ISBN 978-0-596-15806-4.
- Summerfield, Mark (2009). *Programming in Python 3* (2nd ed.). Addison-Wesley Professional. ISBN 978-0-321-68056-3.
- Ramalho, Luciano (May 2022). *Fluent Python* (<https://www.thoughtworks.com/insights/books/fluent-python-2nd-edition>). O'Reilly Media. ISBN 978-1-4920-5632-4.

External links

- Official website (<https://www.python.org/>) 
- Python documentation (<https://docs.python.org/3/>)
- The Python Tutorial (<https://docs.python.org/3/tutorial/>)
- CultRepo (28 August 2025). *The Story of Python and how it took over the world | Python: The Documentary* (<https://www.youtube.com/watch?v=GfH4QL4VqJ0>). Retrieved 22 July 2026 – via YouTube.

Retrieved from "[https://en.wikipedia.org/w/index.php?title=Python_\(programming_language\)&oldid=1365917903](https://en.wikipedia.org/w/index.php?title=Python_(programming_language)&oldid=1365917903)"